

MONASH
University

Machine Learning for Forecasting Hazardous Air Quality Days in Indian Cities

Jaykumar Vinodbhai Chauhan
Student ID: 34710280

Master of Mathematics
MTH5000 Masters Research Project

Supervisor: Dr Tianhai Tian
School of Mathematics, Monash University

October 17, 2026

Abstract

Air pollution in Indian cities regularly reaches hazardous concentrations, and a forecast is useful only if it arrives before the hazardous day does. This project forecasts daily PM2.5 one to three days ahead at a Delhi monitor and evaluates the forecasts as a warning system rather than by average error alone. The central finding is that the alarm budget a decision maker accepts dominates the choice of model. Across ten specifications, from simple benchmarks and a classical ARIMA model with annual Fourier terms to three machine learning regressors and a recurrent network, the spread in hit rate at a fixed alarm budget is about five percentage points, while moving the budget from forty to one hundred alarms a year moves the hit rate by fifty. Machine learning earns its place only partially. The classical model beats every machine learning regressor at every horizon, a result strengthened rather than weakened by a sensitivity analysis on the cleaning threshold, and the three deep learning models that do win, a GRU, an LSTM and a transformer each tested against it directly on supervisor feedback, win clearly only at one day ahead, each reported as a range across ten seeds rather than a single favourable one; the LSTM beats the GRU at every horizon despite carrying more parameters, the opposite of what motivated preferring the smaller network in the first place, and the transformer, added only after a second round of feedback questioned the small-data argument against it, matches or beats both recurrent networks at every horizon and carries the strongest formal significance against ARIMA of the three at one day ahead. Adding observed wind speed and temperature narrows the one-day gap further but leaves two and three days a statistical tie, and the extra accuracy moves the achievable hit rate at a fixed alarm budget by at most a point, reinforcing rather than overturning the central finding above. Two real defects in the source monitor, a clipped instrument ceiling and a miscalculated coverage statistic, were diagnosed and corrected before modelling began. The recommendation is to design a warning system around its alarm budget, treating the forecasting model as a secondary refinement.

Contents

Abstract **i**

List of Symbols **vii**

1 Introduction **1**

1.1 Research Background 1

1.2 Research Problem 1

1.3 Research Objectives 2

1.4 Research Questions 2

1.5 Research Significance 3

1.6 Structure of the Report 3

2 Literature Review **4**

2.1 Air Quality Forecasting: Challenges and Characteristics 4

2.2 Classical and Machine Learning Methods for Time Series Forecasting 4

2.3 Deep Learning Methods for Time Series Forecasting 6

2.3.1 The Deep Learning Landscape for Time Series 6

2.3.2 Recurrent Neural Networks and the Vanishing Gradient Problem 8

2.3.3 The Gated Recurrent Unit 8

2.4 Machine Learning Applications to Delhi and Indian Air Quality 9

2.5 Forecast Verification as a Warning System 10

2.6 Data Quality in Reference Air Quality Monitors 11

2.7 Research Gap and Chapter Summary 11

3 Methodology **13**

3.1 Research Design . . . 13
3.2 Data Collection and Station Selection . . . 13
3.3 Data Preprocessing . . . 14
3.3.1 Data Cleaning: Two Defects in the Source Data . . . 14
3.3.2 Seasonality Analysis . . . 16
3.3.3 Feature Construction . . . 18
3.3.4 Weather Covariates . . . 18
3.4 Classical and Machine Learning Methods . . . 19
3.4.1 Classical Benchmark: ARIMA with Fourier Terms . . . 20
3.4.2 Penalised Linear Regression . . . 21
3.4.3 Random Forest . . . 22
3.4.4 Histogram Gradient Boosting . . . 23
3.5 Deep Learning Methods . . . 24
3.5.1 The Gated Recurrent Unit: Architecture and Training . . . 24
3.5.2 The Long Short-Term Memory Network: Architecture and Training . . . 26
3.5.3 The Transformer: Architecture and Training . . . 28
3.6 Rolling-Origin Evaluation . . . 30
3.7 Evaluation Measures . . . 30
3.8 Statistical Comparison of Forecasts . . . 32
3.9 Reproducibility . . . 33
3.10 Chapter Summary . . . 33

4 Results and Analysis **35**
4.1 Forecast Accuracy . . . 35
4.1.1 Does Weather Change the Answer? . . . 38
4.2 Two Arms of the Same Question . . . 41
4.3 The Decision Threshold . . . 43
4.4 Chapter Summary . . . 45

5 Discussion **46**
5.1 The Operational Question Dominates the Modelling Question . . . 46
5.2 How Large Is a Difference Between Models? . . . 47

5.3 Limitations 48
5.4 Chapter Summary 53

6 Conclusion **54**

6.1 Summary of the Study 54
6.2 Main Conclusions 54
6.3 Future Work 56
6.4 Chapter Summary 57

List of Figures

3.1 The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage. 15

3.2 The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days. 16

3.3 No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero. 17

3.4 The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test. 18

4.1 Table 4.1 as a dot plot, one row per model, one panel per horizon, the vertical line at 1.000 marking persistence. None of the three seeded networks has a single dot: each one's ten seeds are shown as a horizontal range bar rather than a point estimate. Climatology and the seasonal-naive benchmark are omitted here, both well outside this axis range at one day ahead; their numbers are unchanged in Table 4.1. 36

4.2 Table 4.6 extended to all three horizons: the hit-rate advantage of forecast-then-threshold over its matched classifier, by alarm budget, one line per matched pair. Above the zero line forecast-then- threshold wins; below it the classifier does. The lines start mostly above zero at a budget of 40 and drift toward it, and below it, as the budget loosens and the horizon lengthens, the same narrowing the text above describes as counts rather than as a curve. 43

4.3 The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility. 44

List of Tables

3.1 The analysis dataset after cleaning. 14

3.2 Cleaning rules applied, and the basis for each. 15

4.1 Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The three networks, the GRU, the LSTM and the transformer, are each given as the range over ten initialisation seeds; they are the only models whose result depends on one. . . . 35

4.2 Diebold-Mariano test of each network against the ARIMA benchmark (Section 3.8), run once per seed and given as the range across all ten. A negative statistic favours the network at that seed; a positive one favours ARIMA. The last column counts seeds reaching $p < 0.05$, two-sided. 37

4.3 Diebold-Mariano test of ARIMA against each point-forecast machine learning regressor (Section 3.8). A negative statistic favours the machine learning model; a positive one favours ARIMA. An asterisk marks $p < 0.05$, two-sided. 39

4.4 Change in rMAE from adding weather covariates, same rolling-origin protocol and same reference as Table 4.1. Negative is an improvement. The network's weather row is given as its own absolute range over ten seeds, to be read against its row in Table 4.1. 39

4.5 Best achievable hit rate within an alarm budget, before and after adding the weather models as candidates, averaged over the five network seeds. The two rows at each horizon are scored on identical origins; the only difference is which models are allowed to be the best one. 41

4.6 Forecast-then-threshold against direct classification, matched by base algorithm, at one day ahead. Higher is better; bold marks the better arm in each column. . 42

4.7 Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying. 43

List of Symbols

Grouped by the section each symbol is first used in, rather than alphabetically, since the report moves between several method families each with its own standard notation, and a handful of letters are reused for genuinely different quantities across them. Where that happens, the other meaning is named rather than left for the reader to notice.

Symbol	Meaning	Section
General notation, used throughout		
n	number of observations or scored rows in the quantity at hand	
y_t	the series value at time t; on the log scale unless stated otherwise	
e_i, e_t	a forecast error, $\hat{y} - y$	
h	forecast horizon, 1, 2 or 3 days; distinct from the GRU's hidden state h_t below	3.5.1
Seasonality Analysis		
ω_j	Fourier frequency, $2\pi j/n$	3.3.2
$I(\omega_j)$	periodogram value at frequency ω_j	3.3.2
$\hat{\rho}_k$	sample autocorrelation at lag k; k here is reused later as the AIC parameter count, the Fourier harmonic index, and the Diebold-Mariano lag index, each unrelated to this one	3.3.2
$\bar{y}$	sample mean	3.3.2
F	one-way ANOVA test statistic; distinct from the boosting ensemble F_m below	3.3.2
n_g, $\bar{y}_g$	size and mean of day-of-week group g	3.3.2
H	Kruskal-Wallis test statistic; distinct from the GRU's hidden size H below	3.3.2
R_g	sum of ranks in group g	3.3.2
ARIMA with Fourier Terms		

continued on next page

(List of Symbols, continued)

Symbol	Meaning	Section
B	backshift operator, $By_t = y_{t-1}$; distinct from the random forest's tree count B below	3.4.1
$\phi(B)$, $\theta(B)$	autoregressive and moving-average polynomials	3.4.1
$\phi_1, \ldots, \phi_p$; $\theta_1, \ldots, \theta_q$	AR and MA coefficients	3.4.1
p, d, q	ARIMA order; d here is the differencing order, distinct from the GRU's channel count d below	3.4.1
c	constant term	3.4.1
ε_t	white-noise innovation	3.4.1
x_t	exogenous regressor vector at time t (the Fourier terms)	3.4.1
k	here, the number of free parameters in the AIC formula; reused immediately below as the Fourier harmonic index, and again in the ACF and Diebold-Mariano sections as a lag index	3.4.1
$\hat{L}$	maximised likelihood	3.4.1
K	number of Fourier harmonic pairs (order), 4 for ARIMA and the tree/ridge feature table, 2 for the GRU	3.4.1
α_k, β_k	sine and cosine coefficients at harmonic k; β_k is unrelated to ridge's coefficient vector β below	3.4.1
m, s^2	forecast mean and variance on the log scale, used in the back-transform	3.4.1
Ridge Regression		
X	feature matrix	3.4
β	coefficient vector; distinct from the Fourier coefficients β_k above	3.4
λ	ridge penalty parameter	3.4
I	identity matrix	3.4
$J(\beta)$	the penalised least-squares objective	3.4
Random Forest		
B	number of trees; distinct from ARIMA's backshift operator B above	3.4
$T_b(x)$	prediction of the b-th tree	3.4
$\hat{f}(x)$	forest prediction, the average of $T_1, \ldots, T_B$	3.4
ρ, σ^2	pairwise correlation and variance of the individual trees	3.4
j, s	a candidate split's feature and threshold	3.4

continued on next page

(List of Symbols, continued)

Symbol	Meaning	Section
R_1, R_2	the two regions either side of a split	3.4
Gradient Boosting		
$F_m(x)$	the ensemble after m boosting rounds; distinct from the ANOVA statistic F above	3.4
$h_m(x)$	the tree fitted at round m	3.4
$L(y, F)$	loss function; reused with a different signature, $L(e)$, in the Diebold-Mariano test below	3.4
r_{im}	pseudo-residual for row i at round m	3.4
ν	shrinkage (learning rate)	3.4
M	number of boosting rounds	3.4
γ	the ensemble's constant initial prediction	3.4
The Gated Recurrent Unit		
h_t	hidden state at time t; distinct from the forecast horizon h used everywhere else in this report, the one notational collision in it worth flagging on its own	3.5.1
z_t, r_t	update and reset gates	3.5.1
$\tilde{h}_t$	candidate hidden state	3.5.1
W_z, W_r, W_h; U_z, U_r, U_h	input-to-hidden and hidden-to-hidden gate weight matrices	3.5.1
b_z, b_r, b_h	gate biases (each trained as two halves in the implementation used; Section 3.5.1 explains why)	3.5.1
σ	logistic sigmoid; distinct from the variance σ^2 used in the random forest and ARIMA sections above	3.5.1
$\odot$	elementwise (Hadamard) product	3.5.1
H	number of hidden units, 24 throughout; distinct from the Kruskal-Wallis statistic H above	3.5.1
d	number of input channels, 6 (plain) or 10 (with weather); distinct from ARIMA's differencing order d above	3.5.1
The Long Short-Term Memory Network		
c_t, $\tilde{c}_t$	cell state and candidate cell state at time t, the mechanism absent from the GRU above	3.5.2
f_t, i_t, o_t	forget, input and output gates; i_t collides with the row index i used in the Gradient Boosting block above, the gate's standard name in the literature rather than a notation this project chose	3.5.2

continued on next page

(List of Symbols, continued)

Symbol	Meaning	Section
$W_{f,i,o,c}$	input-to-hidden gate weight matrices, one quartet rather than the GRU's triple	3.5.2
$U_{f,i,o,c}$	hidden-to-hidden gate weight matrices, same quartet pattern	3.5.2
b_f, b_i, b_o, b_c	gate biases, same two-halves training convention as the GRU's	3.5.2

h_t, σ, $\odot$, H and d are reused unchanged from the GRU block above.

The Transformer

d_{model}	embedding width, 16 throughout; distinct from the GRU and LSTM's hidden size H, since this is not a recurrent state	3.5.3
n_{head}	number of attention heads, 2 throughout	3.5.3
d_{ff}	feed-forward sublayer width, $4d_{model} = 64$ throughout	3.5.3
pos, i	position index within the 30-day window and embedding-pair index in the positional encoding; i here is unrelated to the row index i used in the Gradient Boosting block above	3.5.3
PE	sinusoidal positional encoding, added to the input embedding before the first layer	3.5.3
Q, K, V	query, key and value matrices, each a learned projection of the embedded window	3.5.3
W^Q, W^K, W^V	the three projection matrices producing Q, K, V	3.5.3
Z, Z', Z''	the embedded window and its state after each encoder sublayer	3.5.3
W_1, W_2, b_1, b_2	the feed-forward sublayer's two linear layers; unrelated to the GRU and LSTM's own gate weights above	3.5.3

d is reused unchanged from the GRU and LSTM blocks above, the number of input channels.

Evaluation Measures

TP, FP, FN, TN	true/false positive/negative counts at a given decision threshold	3.7
n_{train}	number of training observations	3.7

Statistical Comparison of Forecasts (Diebold-Mariano Test)

$e_{1,t}, e_{2,t}$	forecast errors of the two models being compared, at origin t	3.8
$L(e)$	loss applied to a forecast error; $\lvert e \rvert$ throughout this report	3.8

continued on next page

(List of Symbols, continued)

Symbol	Meaning	Section
d_t	loss differential, $L(e_{1,t}) - L(e_{2,t})$; unrelated to ARIMA's differencing order d above	3.8
$\bar{d}$	mean loss differential	3.8
DM, DM*	the Diebold-Mariano statistic and its Harvey-Leybourne-Newbold small-sample correction	3.8
$\hat{\gamma}_k$	sample autocovariance of d_t at lag k	3.8

Chapter 1

Introduction

1.1 Research Background

Air pollution in Indian cities regularly reaches concentrations classified as hazardous to human health. Fine particulate matter small enough to be classified as PM2.5 penetrates deep into the respiratory system, and sustained epidemiological evidence links exposure to it with increased cardiopulmonary and all-cause mortality [Pope and Dockery, 2006, Chen and Hoek, 2020], which is the reason a health threshold rather than an arbitrary concentration is the event this project forecasts. The public response to such episodes depends on knowing that a severe day is coming before it arrives: a forecast issued on the morning of a hazardous day is of limited value, while one issued a day or more ahead can change behaviour, whether through health advisories, restrictions on outdoor activity, or school closures. Daily pollution series carry structure that makes such forecasting feasible in principle: persistence from one day to the next, a strong annual cycle driven by meteorology and seasonal agricultural burning, and dependence on recent local conditions.

My interest in this question is also personal. Having grown up in India, air pollution was not an abstract statistic but part of ordinary life: seasonal smog that turned daylight hazy, and days when going outside meant a sore throat. Whether those days can be predicted reliably enough to be worth warning people about is a question I have a genuine stake in.

1.2 Research Problem

Existing machine learning studies of Indian air quality (reviewed in Chapter 2) report forecast accuracy by average error, and do not decompose that accuracy by the operating point at which a warning would actually be raised. A model with respectable average error can still be a poor warning system if it systematically misses the extreme days that are the only ones anyone cares

about, and the reverse also holds: two models with very different average error can produce nearly identical warning performance once a decision threshold is applied. Separately, the project this report was proposed under assumed that daily pollution series carry a weekly cycle alongside the annual one, driven by traffic and industrial activity, and treated reconciling two seasonal periods as a central modelling difficulty. Neither assumption is safe to carry into the modelling without testing it directly, and both are tested in this report.

1.3 Research Objectives

1. To develop and evaluate a suite of classical, machine learning, and deep learning models for forecasting daily PM2.5 concentrations one to three days ahead at a single, well-characterised Indian monitoring station, under an evaluation protocol that cannot leak future information into past forecasts.

2. To compare two formulations of the hazardous-day warning problem: forecasting the concentration and applying a health threshold to it, against classifying the exceedance event directly.

3. To quantify forecast skill as a warning system, using hit rate and false alarm rate under a varying decision threshold, rather than relying on average error alone, and to establish how much of that skill is attributable to the choice of model versus the choice of decision threshold.

1.4 Research Questions

1. How accurately can daily concentrations be forecast one to three days ahead by machine learning, relative to classical and naive benchmarks?

2. Is it better to forecast the concentration and then apply a threshold, or to classify the exceedance event directly?

3. Considered as a warning system, what hit rate and false alarm rate does each approach achieve, and how does the decision threshold trade them off?

The answer to the third question turns out to dominate the other two, and the report is organised accordingly.

1.5 Research Significance

The practical significance of this work is in the design of hazardous air quality warning systems: the finding that the decision threshold dominates the choice of model, developed in Chapter 4 and Chapter 5, implies that the alarm budget a health authority is willing to raise is a more consequential design decision than the forecasting model behind it. The methodological significance lies in applying a leakage-tested rolling-origin evaluation, rather than a single train-test split, to a model suite spanning classical statistics, machine learning, and a deep recurrent architecture, on a dataset small enough that the comparison between method families is itself informative about where each is reliable.

1.6 Structure of the Report

Chapter 2 reviews the literature on air quality forecasting, classical and machine learning methods for time series, deep learning methods for time series, and forecast verification as a warning system. Chapter 3 describes the data, the two defects found in it, and the full mathematical specification of every model fitted, organised into machine learning methods and deep learning methods as the chapter's two central sections, followed by the rolling-origin evaluation protocol. Chapter 4 presents the results against the three research questions. Chapter 5 interprets those results and states the project's limitations. Chapter 6 concludes and proposes future work.

Chapter 2

Literature Review

2.1 Air Quality Forecasting: Challenges and Characteristics

Daily particulate matter series present a specific set of statistical challenges that distinguish them from the time series most forecasting methods were developed against. The distribution of PM2.5 concentrations is strongly right skewed, with a small number of extreme episodes, rather than normally distributed around a stable mean; the series is strongly seasonal at the annual scale; and the quantity of practical interest, whether a day exceeds a health threshold, is a minority class, typically under a third of days even at a severe site. These three properties motivate working in logarithms rather than on the raw concentration scale (Section 3.4.1), motivate testing rather than assuming the seasonal structure present in the series (Section 3.3.2), and motivate evaluating forecasts as a warning system rather than by a single average-error statistic (Section 2.5).

2.2 Classical and Machine Learning Methods for Time Series Forecasting

The classical approach to a series with this structure is an autoregressive integrated moving average (ARIMA) model, in which a series is differenced to stationarity and then modelled as a linear combination of its own past values and past forecast errors, following the identification and estimation methodology set out by Box et al. [2008]. Seasonal variants extend this by one additional seasonal period, but a series with two seasonal periods, for instance a weekly cycle and an annual one, cannot be accommodated by a single seasonal ARIMA order, which is one of the two assumptions this project tests directly rather than carrying over unexamined. The order itself is conventionally chosen by an information-criterion search over a bounded grid rather than fixed in advance, an approach formalised for routine use by Hyndman and Khandakar

[2008]; this project follows the same principle, an explicit grid scored by the Akaike information criterion rather than a single order asserted without justification (Section 3.4.1). Where the seasonal pattern itself is complex, for instance more than one period or a period that does not divide the sampling interval evenly, Fourier terms offer a compact way to carry it as a small number of exogenous regressors rather than as a large seasonal order; De Livera et al. [2011] develop this idea within an exponential smoothing framework for series with multiple seasonal periods, and the same device, harmonic regressors at a fixed set of frequencies, is the one this project uses to carry the annual cycle in an ARIMA model's exogenous term (Section 3.4.1).

Machine learning offers an alternative that does not assume a linear relationship between a forecast and its predictors. Penalised linear regression controls the variance of an ordinary least squares fit by shrinking its coefficients, which matters when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are [Hoerl and Kennard, 1970]. Random forests and gradient boosting both build on classification and regression trees [Breiman et al., 2017], which recursively partition the feature space to minimise within-region error, but combine many trees by two different mechanisms. A random forest averages trees fitted independently to bootstrap-resampled data, a technique named bootstrap aggregating, or bagging, and shown to reduce the variance of an unstable predictor such as a tree without changing its bias [Breiman, 1996, 2001]. Gradient boosting instead fits an ensemble additively, each new tree correcting the residual error of the ensemble so far; this traces to boosting algorithms that combine many weak learners into a strong one by reweighting misclassified examples [Freund and Schapire, 1997], which Friedman [2001] reformulated as stagewise functional gradient descent, the form implemented by the histogram-based estimator used in this project (Section 3.4) and, in a widely used alternative implementation not used here, by XGBOOST [Chen and Guestrin, 2016]. None of these three machine learning methods assumes stationarity, a seasonal order, or a linear functional form, which is their appeal relative to ARIMA; their cost is that they fit the function empirically from a necessarily finite sample, and Makridakis et al. [2020] report, across 100,000 series in the M4 forecasting competition, that this flexibility does not reliably translate into an advantage over classical benchmarks on series of a few thousand observations, which is the order of magnitude available in this project. This was not a new finding when M4 confirmed it at scale: Makridakis et al. [2018] had already reported the same pattern on the smaller M3 dataset and argued that the forecasting community had adopted machine learning methods faster than the evidence for their superiority had accumulated, and Siami-Namini et al. [2018] report a direct head-to-head instance of it, an LSTM failing to beat ARIMA on a set of financial and economic series very different in domain from this project's but identical in being short relative to a deep network's capacity. The picture is not uniformly negative for machine learning, however: in the M5 competition, run on retail demand series with genuine exogenous and hierarchical structure to exploit, gradient boosting methods won decisively [Makridakis et al., 2022]. Read together, M4 and M5 suggest the deciding factor is not machine learning versus classical methods in the abstract but whether the series on offer

gives a flexible method something structural to exploit beyond what a classical model already captures; Chapter 4 revisits this reading directly; a single daily PM2.5 series with no exogenous predictors is closer to the M4 setting than the M5 one, which is the comparison this project's own result in Chapter 4 confirms rather than contradicts. A different strand of this literature asks not which family wins outright but whether they can be combined: Zhang [2003] fit a neural network to an ARIMA model's residuals and report a gain over either model alone. This project does not pursue a hybrid, evaluating the two families side by side instead, precisely so that the comparison in Chapter 4 is informative about which family suits this series rather than obscured by a blend of both.

Whichever family is used, the comparison is only as trustworthy as the procedure used to test it, a methodological point this project treats as seriously as the choice of model. Tashman [2000] reviews decades of out-of-sample forecasting evaluations and argues that a single train-test split, still common practice at the time, understates how much accuracy varies across time periods; Hewamalage et al. [2023] revisit the same concern two decades later and document the same pitfalls still occurring routinely in applied forecasting work, including exactly the single-split optimism this project found and corrected in Section 3.6. The rolling-origin design adopted here (Section 3.6) is this project's answer to both papers' shared complaint. The accuracy measure itself is a related choice: Hyndman and Koehler [2006] propose the mean absolute scaled error specifically because percentage errors are undefined when a series can be zero and unstable when it is close to zero, and this project reports it alongside the relative mean absolute error introduced in Section 3.7 for exactly the reason their paper gives, that a scaled error's apparent improvement can reflect a change in the series' own volatility rather than a change in forecast skill, which is precisely what was found when the single split was replaced by the rolling-origin harness.

2.3 Deep Learning Methods for Time Series Forecasting

2.3.1 The Deep Learning Landscape for Time Series

Before narrowing to the single architecture this project fits, it is worth stating why the others on offer were not chosen, since "we used a recurrent network" is a much weaker claim without the alternatives it was weighed against. The simplest deep architecture is the multilayer perceptron (MLP), a feedforward network shown to approximate any continuous function given enough hidden units [Hornik et al., 1989], applied to forecasting since at least the 1990s [Zhang et al., 1998]. An MLP has no sense of sequence order built in: it treats its input as an unordered feature vector, so applying one to a time series means first constructing lagged and rolling-window features exactly as Section 3.3.3 already does for the penalised regression and tree models. An MLP fed that table would not be a meaningfully different model from those already

in the suite; the sequence-aware architectures below exist precisely to avoid depending on which lags a human chose to hand-engineer.

Convolutional neural networks (CNNs) take the opposite approach, sliding a learned kernel along the time axis to detect local patterns directly from the raw sequence, originally developed for images [LeCun et al., 1998] and adapted to seasonal time series with trends directly comparable to this project's own series [Liu et al., 2020]. A convolution's receptive field is fixed by its kernel width and depth, so capturing a cycle as long as a year needs either a very deep stack or a very wide kernel; this project's own annual cycle is instead handed to every model directly as a small number of Fourier terms (Section 3.3.3), which removes much of the advantage a CNN's automatic pattern detection would otherwise offer here.

Transformer architectures, outside the time series domain, replace recurrence altogether with self-attention, letting every timestep attend directly to every other one rather than propagating information step by step. Adaptations for multi-horizon time series forecasting do not always make that same substitution outright: the Temporal Fusion Transformer [Lim et al., 2021], for instance, keeps a recurrent layer for local, short-range processing and adds self-attention on top of it specifically for long-range dependence and interpretability, a hybrid rather than a wholesale replacement of recurrence. Either form carries the same practical cost relative to a small GRU: a substantially larger parameter count, tuned and validated on datasets several orders of magnitude larger than the roughly three thousand daily observations available here. The same sample-size argument Section 2.2 makes against flexible machine learning methods on a series this size applies with more force to an architecture built for sequences of many thousands of tokens. Section 3.5.3 and Chapter 4 test this reasoning directly rather than leaving it a citation-backed assumption, once a transformer was actually fitted under the identical protocol used for the two recurrent networks, a step taken only after Dr Tian questioned the premise directly: the series is some four thousand days long counting every evaluation block, which by his own reading of the literature clears the threshold at which a transformer is worth trying. The result does not support the argument above as stated: the fitted transformer beats the GRU at every horizon and is competitive with the LSTM, the same pattern already found below for the GRU-over-LSTM parameter-count argument.

Weighed against all three, a recurrent network is the architecture whose assumptions best match this project's data: it reads the sequence directly, unlike an MLP; it is not limited to a fixed receptive field, unlike a CNN, though that limitation matters less here given the Fourier terms already carry the annual cycle; and, on the a priori parameter-count argument above, it was expected to suit a few thousand observations better than a Transformer would, an expectation Section 3.5.3 tests directly rather than confirms. Within the recurrent family, the remaining choice is between a plain recurrent network and a gated one, which the next two subsections address.

2.3.2 Recurrent Neural Networks and the Vanishing Gradient Problem

A recurrent neural network processes a sequence by maintaining a hidden state that is updated at every timestep as a function of the current input and the previous hidden state, which in principle lets it represent dependence on the arbitrarily distant past without the fixed lag structure a regression feature table imposes, trained by backpropagation through time, a direct extension of the backpropagation algorithm for feedforward networks [Rumelhart et al., 1986]. In practice, a plain recurrent network trained this way suffers from vanishing or exploding gradients as the sequence lengthens: Bengio et al. [1994] show formally that the conditions which let a recurrent network store information robustly over many timesteps are, under gradient descent, precisely the conditions under which the gradient of a distant error with respect to an early input either vanishes or explodes, so that reliably learning long-range dependence and reliably training by gradient descent are in tension with each other. Hochreiter and Schmidhuber [1997] address this with the long short-term memory (LSTM) architecture, a memory cell and three multiplicative gates controlling what is written to, read from, and forgotten by that cell, at the cost of a comparatively large number of parameters for a short daily series.

2.3.3 The Gated Recurrent Unit

The gated recurrent unit (GRU) simplifies the LSTM to two gates and no separate memory cell, while retaining the property that gradients can flow across many timesteps without vanishing [Cho et al., 2014]. Its full specification, and the specific architecture used in this project, is given in Section 3.5.1. The GRU's a priori appeal for a daily series with roughly three thousand observations was its smaller parameter count relative to an LSTM, on the reasoning that fewer parameters matter more on a dataset this size than the slightly larger representational capacity an LSTM's extra gate and cell state provide. Section 3.5.2 and Chapter 4 test that reasoning directly rather than leaving it a citation-backed assumption, once an alternative architecture was actually fitted under the identical protocol: an LSTM outperforms the GRU at every horizon in this project's own results, so the parameter-count argument, a defensible prior, is not what this project's own evidence supports. Lim and Zohren [2021] survey the broader move of deep learning into time series forecasting of which the recurrent, convolutional and attention-based families in Section 2.3.1 are all part, and note that the gain such architectures offer over classical methods depends heavily on how much data and exogenous structure a given forecasting problem provides, the same conditioning factor Section 2.2 identifies as deciding whether machine learning or a classical benchmark wins on a series of a given size; this survey, and the general account of deep learning's strengths and limits given by LeCun et al. [2015], frame the recurrent-network results in Chapter 4 as an instance of a known pattern rather than an isolated one. A practical limitation specific to recurrent architectures, and directly relevant to this project's own series, is that they require a value at every timestep and cannot represent a missing

observation natively; Che et al. [2018] study exactly this problem for multivariate clinical time series and adopt the same device used in Section 3.5.1, carrying the last observed value forward alongside an explicit missingness indicator, as a principled rather than ad hoc way to hand a recurrent network a gap in its input.

2.4 Machine Learning Applications to Delhi and Indian Air Quality

Delhi's air pollution problem is well characterised at the level of sources and policy before any forecasting model is applied to it. Guttikunda et al. [2014] review the emission inventory across major Indian cities and identify vehicular traffic, industry, and seasonal biomass burning as the dominant contributors, the last of which is the specific mechanism behind the stubble burning season discussed below. Satellite-based studies independently quantify that mechanism's contribution to Delhi's severe episodes: Cusworth et al. [2018] attribute a substantial share of Delhi's post-monsoon particulate spikes to agricultural fires further north, and Jethva et al. [2019] connect the same fires to reduced regional air quality using more than a decade of satellite fire counts. None of these three studies forecasts the series; they establish the physical mechanism that a purely statistical model, such as every model in this project, can only infer indirectly from the series' own seasonal structure, which is worth stating plainly as a limitation of the approach taken here.

Machine learning applied specifically to Delhi PM2.5 is not a new proposal. Masood and Ahmad [2020] fit two models, a support vector machine and an artificial neural network, to predict Delhi's PM2.5 concentrations from meteorological and pollutant covariates and report that the neural network is the more accurate of the two; their inputs are exogenous measurements at the forecast target day itself rather than the series' own lagged history, which is a different forecasting problem from the one addressed here, where the only inputs available at origin are the series' own past. Singh and Srivastava [2025] narrow the problem to Delhi's autumn stubble burning season, when biomass burning in neighbouring states drives the city's most severe episodes and, by their own account, its air pollutant levels peak in October and November specifically. Their paper combines two analyses rather than one: a health-risk assessment of stubble-season exposure using a dedicated health-risk model, and, separately, a comparison of three machine learning models, an artificial neural network, a support vector machine, and a random forest, for PM2.5 forecasting, with the random forest the most consistent of the three. The health-risk analysis and the forecasting comparison sit alongside each other rather than one inside the other: the forecasting models are themselves still scored by the same average-error measures, R^2, RMSE and MAE, that this project argues are insufficient on their own, not by a health-weighted criterion. Neither paper evaluates its forecasts as a warning system in the sense

used here, with an explicit alarm budget and a false alarm rate reported alongside the hit rate; both report accuracy on held-out data without decomposing it by the operating point at which an alarm would actually be raised. That gap, between forecast accuracy and warning system performance, is the space this project's third research question occupies.

The finding that a classical benchmark beats every machine learning model in this project's own results (Chapter 4) is not an anomaly against the wider forecasting literature discussed in Section 2.2. The mechanism Makridakis et al. [2020] propose, that a few thousand observations is a small sample for a flexible learner relative to the number of parameters a modern architecture can fit, applies directly to a project with roughly three thousand daily observations. The negative result for machine learning at two and three days ahead in Chapter 4 is therefore the expected outcome under that literature rather than a surprise specific to this dataset.

2.5 Forecast Verification as a Warning System

The distinction this project draws between forecast accuracy and warning system performance, hit rate, false alarm rate, and the trade-off between them as the decision threshold moves, follows the verification framework standard in operational meteorology [Jolliffe and Stephenson, 2012]. That literature developed around exactly the situation faced here: a forecast of a rare, high-impact event, where the average error of a model is a poor guide to its value as a warning, and where a contingency table decomposed by decision threshold is more informative than any single accuracy score. Its central methodological point, that verification measures should be chosen to match the decision a forecast supports rather than borrowed by default from a different problem, motivated reporting hit rate and false alarm rate as a pair in Section 3.7 rather than a single skill score. The critical success index reported alongside them, the fraction of forecast or observed events that were correctly forecast, with correct quiet days excluded from both numerator and denominator, was introduced by Schaefer [1990] specifically to avoid a statistic inflated by the large number of quiet days that dominate a rare-event problem such as this one, where fewer than a third of days exceed the health threshold.

That same imbalance, a minority of hazardous days against a majority of quiet ones, is the reason this project never reports plain classification accuracy as a warning-system measure. He and Garcia [2009] survey the broader problem of learning from imbalanced data and document how a classifier can score highly on overall accuracy by predicting the majority class essentially always, a failure mode this project's own alarm-budget framing (Section 3.7) is designed to make visible rather than hide, since an alarm budget of zero would trivially achieve a hit rate of zero rather than a flattering accuracy figure. Saito and Rehmsmeier [2015] make the related point that a receiver operating characteristic curve, a common default for a binary classifier, can look similarly optimistic on imbalanced data because it is insensitive to a large true-negative count that costs nothing to accumulate; a precision-recall-style decomposition, of which the

hit rate and false alarm rate pair used throughout this report is a close relative, is the more informative choice for exactly this kind of problem.

2.6 Data Quality in Reference Air Quality Monitors

The data quality problems documented in Section 3.3.1 are not unique to this station. Jiang et al. [2023] address the companion problem to the one found here: reference monitors occasionally report physically impossible negative PM2.5 concentrations, arising from instrument noise near zero rather than a true measurement, and propose a defensible statistical threshold below which a reading should be treated as invalid rather than retained or naively clipped to zero. Their approach and the one used in Section 3.3.1 for the clipped ceiling at 1990 share a method rather than a target: both treat an implausible value as a data quality question to be settled by evidence, specifically its relationship to the plausible range and its recurrence pattern, rather than by a rule of thumb applied uniformly. Considered together, artefacts at both ends of the distribution suggest that reference-grade air quality monitors carry systematic measurement pathologies that a forecasting study must characterise explicitly rather than assume away.

A third data quality question, separate from an implausible value being present, is a plausible value being absent. Little and Rubin [2019] set out the standard framework for reasoning about why data are missing, missing completely at random, missing at random given other observed information, or missing in a way that depends on the unobserved value itself, and show that the appropriate treatment differs sharply between the three. The coverage rule adopted in Section 3.3.1, excluding a day when too few readings contributed to its mean, is a judgement that the gaps here are closer to the first two categories than the third: nothing in the station's own account of why a reading is missing suggests that a reading is more likely to be dropped specifically when the true concentration is unusually high or low, as opposed to for reasons such as instrument maintenance that are unrelated to the pollution level itself. The remaining short gaps that survive the coverage rule are carried forward rather than discarded, using the same forward-fill-plus-indicator treatment discussed for the recurrent network in Section 3.5.1, which is defensible under the same assumption.

2.7 Research Gap and Chapter Summary

This chapter has covered five bodies of literature: the statistical characteristics of air quality series that motivate the methodological choices in Chapter 3; classical and machine learning methods for time series forecasting, their shared lineage through bagging, boosting and regression trees, and why the forecasting-competition literature predicts a flexible learner need not beat a classical benchmark on a small sample, together with the methodology for testing that

comparison honestly, out-of-sample evaluation and scale-free accuracy measures, rather than taking any single reported number at face value; deep learning methods for sequences, the wider landscape of feedforward, convolutional and attention-based architectures weighed against the recurrent family before narrowing to it, the specific architectural reason a GRU rather than an LSTM suits a dataset this size, and the structural limitation, an inability to represent a missing input, that a recurrent architecture carries regardless of which gated variant is chosen; the physical background and Delhi-specific forecasting literature, emission sources, biomass burning, and two prior machine learning studies on this same city's PM2.5, that motivates treating this as a hazard-warning problem rather than a curve-fitting exercise and locates the gap, warning-system evaluation against an explicit alarm budget, that those studies leave open; and the two literatures, forecast verification and air quality monitor data quality, that this project's own findings connect back to directly. Taken together, this literature places the project's contribution precisely. Applying an established model suite to Delhi's air quality problem is not new, and neither is the finding that classical methods hold their ground against machine learning on a modest sample. What the Delhi-specific studies above do not do is evaluate that comparison as a warning system with an explicit, varying alarm budget, using verification methodology from a literature that reaches air quality forecasting only indirectly. Chapter 4 shows why that omission matters: the ranking of models looks different once accuracy is separated from operational usefulness, a finding neither literature taken alone would have anticipated.

Chapter 3

Methodology

3.1 Research Design

This chapter proceeds from data to model to evaluation. Section 3.2 describes the data source and the station-selection process. Section 3.3 describes two defects found in the raw series, the seasonality analysis that overturned an assumption in the approved proposal, and the feature table built from the cleaned series. Section 3.4 and Section 3.5 give the full mathematical specification of the two families of model this project compares, machine learning and deep learning, each covering the method's general form before the specific configuration used here. Section 3.6 specifies the rolling-origin evaluation every model is scored under, and Section 3.7 specifies the accuracy and warning-system measures reported in Chapter 4.

3.2 Data Collection and Station Selection

Daily mean $PM_{2.5}$ concentrations were obtained from OpenAQ. Five candidate monitoring stations within 25 km of central Delhi were compared on data coverage rather than on nominal length of record.

The comparison produced an unexpected constraint. Every station operated by the Central Pollution Control Board returns measurements only from 19 February 2025, approximately eighteen months, although the OpenAQ location metadata advertises a first observation in February 2016. Four independent stations sharing an identical cutoff indicates a systematic limit on what the measurements endpoint serves rather than a coincidence. Only location 8118 returns the full record and it was therefore selected.

This carries a limitation that must be stated plainly. Location 8118 is supplied through AirNow and is a United States diplomatic post monitor rather than part of the Indian official network. The project therefore forecasts at a single well-characterised reference site in Delhi,

which is common in this literature, and the results should not be read as representative of the city as a whole.

The event this project forecasts, a hazardous $PM_{2.5}$ day, is defined against the Central Pollution Control Board's National Air Quality Index [Central Pollution Control Board, 2014], which sets the boundary of its 'Very Poor' category, the one directly below 'Severe', at $121\,\mu g\,m^{-3}$ for the 24-hour mean. Table 3.1 below reports exceedance against this threshold, $121\,\mu g\,m^{-3}$, rather than an arbitrary round number.

Table 3.1: The analysis dataset after cleaning.

Source	OpenAQ location 8118, sensor 23534, provider AirNow
Period	2016-11-09 to 2026-08-27
Calendar days	3579
Usable daily means	3093 (86.4% of calendar)
Longest gap	34 consecutive days
Mean, median, maximum	102.7, 71.7, 896.0
Days above 121	920 (29.7%)

3.3 Data Preprocessing

3.3.1 Data Cleaning: Two Defects in the Source Data

Two problems were identified before any modelling, one of which was silently corrupting the series.

Physically impossible values. The raw series contained five negative concentrations and a maximum of $1990\,\mu g\,m^{-3}$. The value 1990.0 appeared three times identically on unrelated dates, which is the signature of a clipped instrument ceiling rather than a measurement. One occurrence fell on 2 September, during the monsoon, between neighbouring days of 42 and 178.

An incorrect coverage denominator. OpenAQ reports the completeness of each daily aggregate as the observation count divided by an expected count of 24, assuming hourly reporting. The sensor at location 8118 reported half-hourly from 2016 to 2024 and hourly from 2025 onward. The visible symptom is a reported completeness exceeding 100 per cent, which is impossible. The damaging symptom is invisible: a day holding 24 of a genuine 48 readings is scored as fully complete, so a mean computed from half a day of a polluted afternoon enters the series as though it were a daily mean. Days at 50 to 75 per cent true coverage produced implausible values approximately seven times as often as days above 90 per cent, which identifies incomplete days as the mechanism rather than a coincidence.

The correction takes the reporting cadence from the data, using the ninetieth percentile of observation counts within each year, which follows the 2025 change in reporting interval automatically. Recomputed on that basis, 1,085 of the 3,467 raw calendar-linked days fall below 75 per cent true coverage, far more than the 268 days that have no recorded value at all and would be the only ones flagged by a check for whether a value exists rather than how reliably it was measured.

Table 3.2: Cleaning rules applied, and the basis for each.

Rule	Days removed	Basis
Value below zero	5	Mass concentration cannot be negative
Coverage below 50%	270	Against the cadence taken from the data
Value above 1000	7	Judgement; sensitivity analysis in Section 5.3
Total	**282 of 3,375**	

The coverage threshold is set at 50 per cent rather than a stricter value because the longest gap is the binding constraint: at 60 per cent the worst gap rises to 69 days and at 90 per cent to 185 days, either of which would distort estimation of the annual cycle. At 50 per cent the worst gap is 34 days. The exceedance rate moves only from 30.0 to 27.0 per cent across every candidate threshold, which confirms that the filter is not selectively removing the events of interest.

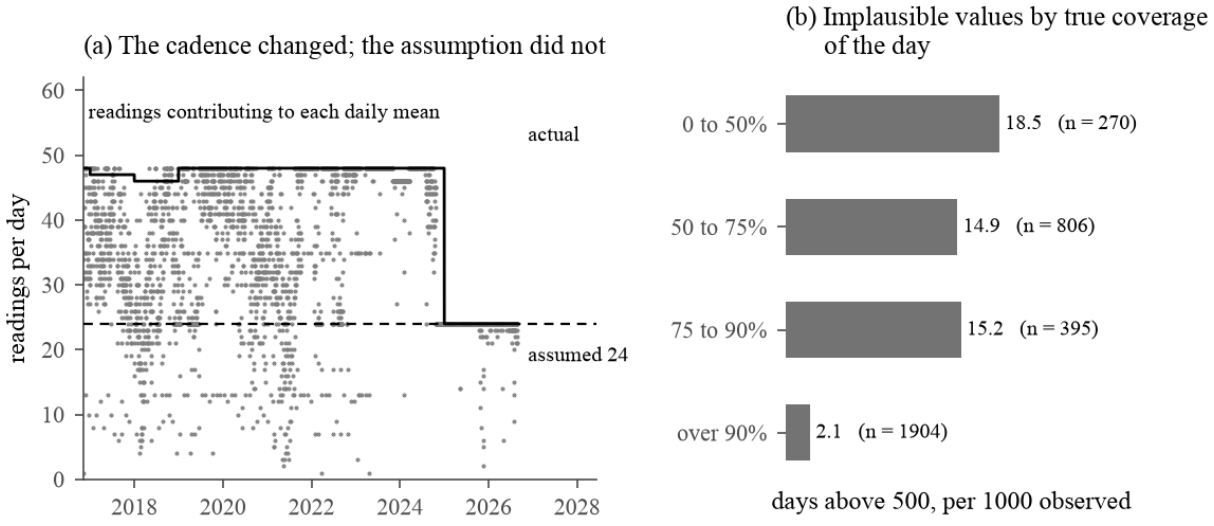


Figure 3.1: The coverage defect. Panel (a) shows the number of readings contributing to each daily mean, with the cadence estimated from the data each year as the solid line and the fixed denominator of 24 as the dashed line. Panel (b) shows the rate of implausible values by true coverage.

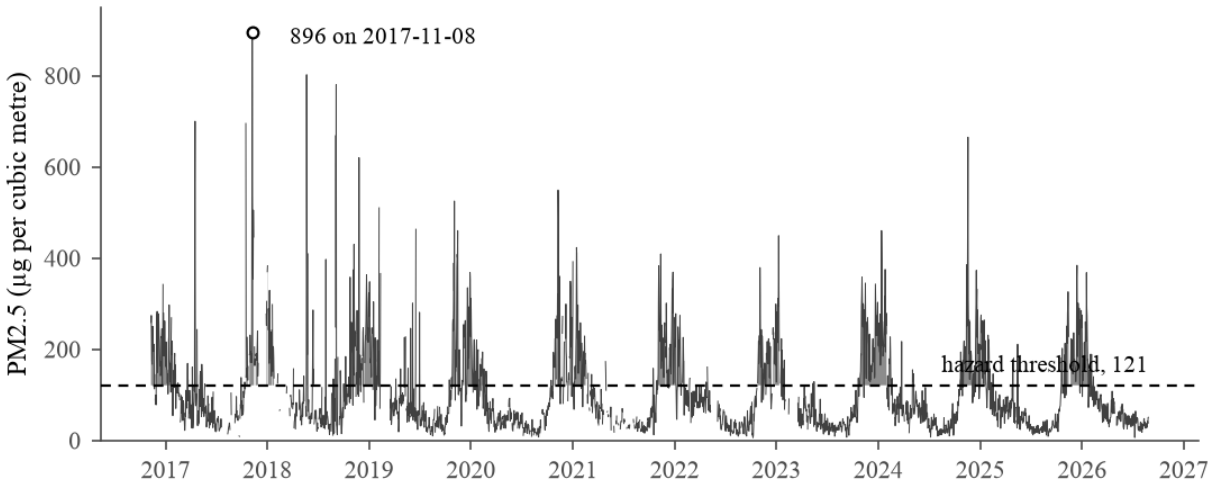


Figure 3.2: The cleaned daily series, with days exceeding the threshold shaded. The episodic structure is the feature the project sets out to forecast: exceedances arrive as multi-day winter episodes rather than as isolated days.

3.3.2 Seasonality Analysis

The approved proposal states that daily pollution series carry a weekly cycle driven by traffic and industrial activity, and treats the coexistence of a weekly and an annual period as the central modelling difficulty, since a seasonal ARIMA accommodates only one seasonal period.

Tested on the cleaned series, the weekly cycle is not present at this station. Three independent tests agree, each asking a different kind of question of the same series: whether its variance concentrates at a weekly frequency, whether a lag-7 dependency survives beyond what the series' own short-range decay would produce anyway, and whether the day of the week carries a mean effect.

The periodogram estimates how a series' variance decomposes across frequency, from the magnitude of its discrete Fourier transform at each Fourier frequency $\omega_j = 2\pi j/n$,

$$I(\omega_j) \;=\; \frac{1}{n}\left[\Big(\sum_{t=1}^{n} y_t \cos(\omega_j t)\Big)^2 \;+\; \Big(\sum_{t=1}^{n} y_t \sin(\omega_j t)\Big)^2\right], \tag{3.1}$$

read as a period of n/j days rather than a frequency. It gives dominant periods of 357.9, 178.9, 188.4, 397.7, 325.4 and 60.7 days, which is the annual cycle and its harmonics, with nothing near seven.

On the deseasonalised logarithm of concentration, the sample autocorrelation at lag k,

$$\hat{\rho}_k \;=\; \frac{\sum_{t=k+1}^{n}(y_t-\bar{y})(y_{t-k}-\bar{y})}{\sum_{t=1}^{n}(y_t-\bar{y})^2}, \tag{3.2}$$

is 0.173 at lag 7, while lag 6 is 0.179 and lag 8 is 0.153, so lag 7 lies on the decay curve rather than above it; the same holds at lags 14 and 21. A genuine weekly effect would show as a spike

at lag 7 breaking that otherwise smooth decay, not merely as some nonzero value, since every short lag is autocorrelated regardless of any seasonal structure at all.

The day of week effect on the same residuals spans 6.5 per cent between the highest and lowest day. A one-way analysis of variance tests whether the seven day-of-week group means differ by more than the scatter within each group predicts,

$$F \;=\; \frac{\sum_{g=1}^{7} n_g (\bar{y}_g - \bar{y})^2 \,/\, (7-1)}{\sum_{g=1}^{7} \sum_{i=1}^{n_g} (y_{gi} - \bar{y}_g)^2 \,/\, (n-7)}, \tag{3.3}$$

giving $F = 1.13$, $p = 0.34$ against an $F_{6,n-7}$ reference distribution, and a Kruskal-Wallis test,

$$H \;=\; \frac{12}{n(n+1)} \sum_{g=1}^{7} \frac{R_g^2}{n_g} \;-\; 3(n+1), \tag{3.4}$$

the same comparison on each observation's rank summed within group g, R_g, rather than on the residual directly, asymptotically χ^2_6 under the null and free of the normal-residual assumption Equation 3.3 makes, gives $p = 0.77$. Three tests built on different assumptions, a spectral decomposition, a linear-model F-test, and a distribution-free rank test, agreeing is what makes the absence convincing rather than an artefact of one test's particular assumptions failing to hold here.

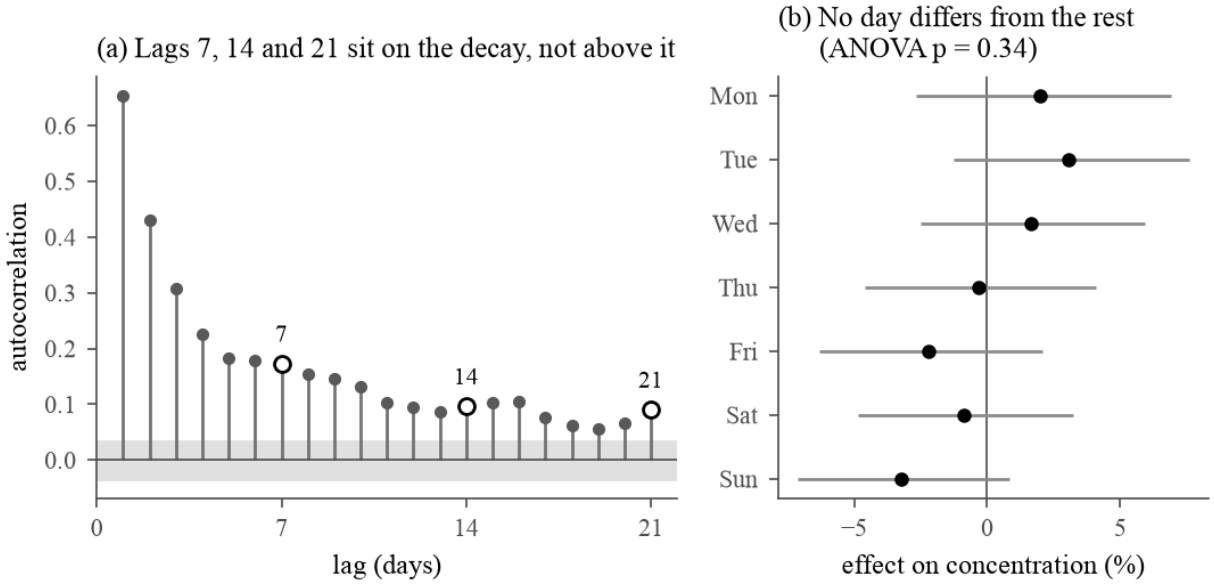


Figure 3.3: No weekly cycle. Lags 7, 14 and 21 are circled and lie on the decay curve rather than above it. Every day of week interval contains zero.

There is a physical reading rather than a null result. A weekly rhythm in urban particulate concentration is a traffic signature, and location 8118 sits in the diplomatic enclave rather than at a roadside site such as ITO or Anand Vihar. Its absence at a site that is not traffic dominated is what one would expect.

The consequence for the specification is direct: the classical benchmark is ARIMA with annual Fourier terms as exogenous regressors and no weekly seasonal order. The specification is simpler, better identified, and empirically justified rather than assumed.

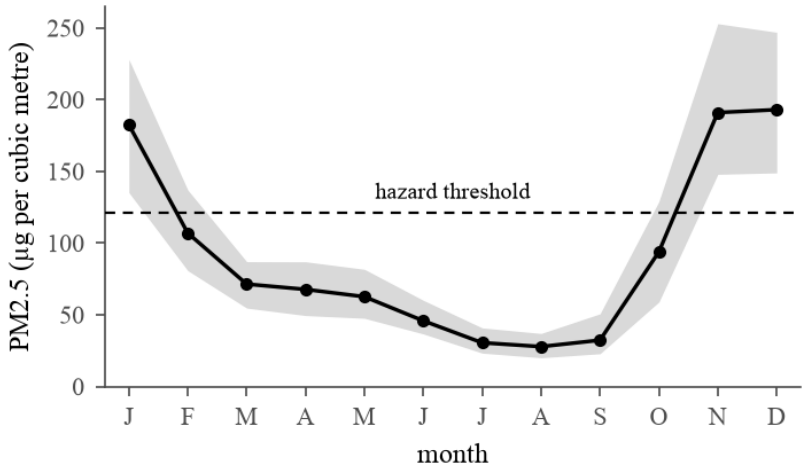


Figure 3.4: The annual cycle, by contrast, is unmistakable: monthly median with the interquartile range shaded. Its strength is what makes the absence in Figure 3.3 credible rather than an artefact of a weak test.

3.3.3 Feature Construction

A supervised table was built with one row per forecast origin. A row indexed by date t carries only functions of observations up to and including day t, and its targets are the concentrations on days $t+1$, $t+2$ and $t+3$. Features comprise lags at 0, 1, 2, 3 and 7 days; first differences; rolling mean, standard deviation and maximum over 3, 7, 14 and 30 days; an anomaly and standardised anomaly against the 30 day norm; exceedance frequency over the last 7 and 30 days; days since the last exceedance; calendar terms; four pairs of annual Fourier terms; and three missingness indicators.

Missing values are never imputed. Gradient boosting accepts them natively and the state space models skip the measurement update at a gap, so the absence is carried as information rather than replaced by a fabricated value. The single exception is the recurrent networks, the GRU and the LSTM, which share the same channel construction and are discussed together in Sections 3.5.1 and 3.5.2.

3.3.4 Weather Covariates

The feature table above is extended with daily surface weather for the same station coordinates, fetched through the `meteostat` package, which aggregates NOAA and GHCN-Daily records. Six variables are available at reasonable completeness: mean, minimum and maximum temperature, wind speed, mean sea level pressure, and precipitation. Each enters the table at its own

value, its value one day earlier, and its three day rolling mean, so that a model can use both the current reading and its recent trend. Every weather column obeys the same rule as every other feature in this table: only observations up to and including day t are used. A real early-warning system would use a numerical weather *forecast* for the days ahead rather than the weather already observed, which would very likely help further, but that is a materially stronger claim about what is knowable at the forecast origin, and this project does not make it.

The weather record carries its own data quality defect, a third one for this project, and it was diagnosed the same way as the two in Section 3.3.1: checked directly rather than assumed. Within the window it covers, the record is close to complete, 99.9 per cent for temperature and 99.6 per cent for wind speed. But that window ends 202 days before the PM2.5 series does: the station's archive is current only to reporting delays of several months, a different failure mode from the two already documented, a hard cutoff rather than scattered gaps. Precipitation is worse, with only 54 per cent coverage even before that cutoff, and is excluded from the recurrent network's channels for this reason, retained only for the tree models, which take a sparse column natively rather than needing it imputed.

That cutoff means a meaningful share of the rolling-origin test period in Section 3.6 carries no weather observation at the forecast origin. Rather than let that pass silently, the comparison in Section 4.1 is checked twice: once over every origin, where the gap is handled exactly as every other missing value in this project is, left as `NaN` for gradient boosting, imputed from the training window for ridge and the random forest, and carried forward with a missingness flag for the recurrent network, and once restricted to origins that fall entirely inside the window where weather is genuinely observed. The two give the same answer, which is what makes the first trustworthy.

3.4 Classical and Machine Learning Methods

Six of the twelve specifications fitted in this project belong to the machine learning family in the broad sense used in Section 2.2: the classical ARIMA benchmark, which is included here for comparison even though it predates the term, and the three explicitly machine learning regressors, each fitted to both the level of concentration and its change from the forecast origin. Every model in this section is fitted to the logarithm of concentration rather than the raw scale. Annual Fourier terms explain 69 per cent of the variance of the logarithm against considerably less on the raw scale, and fitting in logs prevents a model predicting a negative concentration, which has no physical meaning.

3.4.1 Classical Benchmark: ARIMA with Fourier Terms

Let $y_t = \log(\text{PM2.5}_t)$ and let B denote the backshift operator, $By_t = y_{t-1}$. Following the Box-Jenkins formulation [Box et al., 2008], an autoregressive integrated moving average model of order (p,d,q) with exogenous regressors x_t is

$$\phi(B)\,(1-B)^d y_t \;=\; c \;+\; \theta(B)\,\varepsilon_t \;+\; \beta^\top x_t, \tag{3.5}$$

where $\phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ is the autoregressive polynomial, $\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$ is the moving average polynomial, and ε_t is white noise. The order is chosen from an explicit grid of candidate (p,d,q) triples by the Akaike information criterion,

$$\text{AIC} \;=\; -2\ln\hat{L} \;+\; 2k, \tag{3.6}$$

where $\hat{L}$ is the model's maximised likelihood and k its number of free parameters, so the criterion rewards fit while charging a fixed penalty for every additional parameter rather than letting the largest model in the grid win by construction. The order scoring lowest on the training block is $(p,d,q) = (1,1,1)$, AIC 1892.5, with the next best candidate in the grid, $(1,1,2)$, at 1893.6: a gap of 1.1, too small to call $(1,1,1)$ decisively better rather than a more parsimonious specification of roughly equal support, which is the more defensible reading of why it was selected. Equation 3.5 specialises to

$$(1-\phi_1 B)(1-B)\,y_t \;=\; c \;+\; (1+\theta_1 B)\,\varepsilon_t \;+\; \sum_{k=1}^{4}\left[\alpha_k \sin\!\left(\tfrac{2\pi k t}{365.25}\right) + \beta_k \cos\!\left(\tfrac{2\pi k t}{365.25}\right)\right], \tag{3.7}$$

with four pairs of annual Fourier terms ($K = 4$) as the exogenous regressors x_t, carrying the annual cycle established in Section 3.3.2 and deliberately carrying no weekly seasonal order. Fitted on the training block, $\hat{\phi}_1 = 0.581$ and $\hat{\theta}_1 = -0.987$. Both must satisfy $|\phi_1| < 1$ and $|\theta_1| < 1$ for the differenced series to be stationary and the model invertible, the conditions under which an ARMA process has the causal, one-sided representation Box-Jenkins estimation assumes [Box et al., 2008]; the fit satisfies both, although $\hat{\theta}_1$ sits only 0.013 inside its boundary, a near-cancellation between the differencing and the moving average term that is a known signature of a series close to overdifferenced rather than evidence the order search chose badly, since $(1,1,1)$ still scored lowest of every order in the grid. The implementation used here does not constrain the optimiser to this region, `enforce_stationarity=False` and `enforce_invertibility=False` in the SARIMAX call, so satisfying both conditions is a property of the fitted optimum on this data rather than one guaranteed by construction.

That pair of values is a snapshot rather than a constant. The rolling-origin evaluation behind every number in Chapter 4 re-estimates this same fixed order twenty-eight times as the training window expands, every 90 days from 2019 to 2026, and the coefficients move over that span

rather than settling immediately. $\hat{\phi}_1$ ranges from 0.520 at the first refit to 0.591 near the last, comfortably inside the stationarity boundary throughout. $\hat{\theta}_1$ is less comfortable: from 2023 onward it stays within 0.012 to 0.014 of the invertibility boundary, matching the value reported above, but at three of the twenty-eight refits, every one with fewer than 1,700 training days, the fitted value crosses the boundary outright, -1.006, -1.011 and -1.013. The optimiser is not constrained to avoid this, so a forecast is still produced and still scored at those refits exactly as at any other; the single pair reported above describes the model this evaluation settles into for most of its span, not a property holding at every refit within it.

Using trigonometric regressors at a fixed set of frequencies to carry a seasonal pattern inside an otherwise standard model, rather than through a seasonal differencing order, follows the same logic De Livera et al. [2011] apply to exponential smoothing for series with seasonal patterns too complex for a single seasonal order to represent; here the device is used to keep the annual cycle outside the ARIMA order entirely, which is what makes carrying no weekly term in parallel straightforward rather than a second seasonal order fighting the first.

Back-transformation from the log scale is not innocent. If $\log y$ has forecast mean m and forecast variance s^2, then under the model's Gaussian error assumption y is lognormally distributed, and

$$\text{median}(y) = \exp(m), \qquad \text{mean}(y) = \exp\left(m + \tfrac{1}{2}s^2\right). \tag{3.8}$$

Both quantities were produced as separately named models, since a report that back-transforms without stating which one it reports has silently changed what is being forecast. The gap between the two, and specifically the tendency of the median variant to systematically underforecast a right-skewed quantity, is a known bias rather than a property special to this project: Miller [1984] derive the same correction term, $\exp(\frac{1}{2}s^2)$ in Equation 3.8, for curve-fitting contexts generally, as the adjustment needed to recover an unbiased mean from a model fitted on the logarithm of a quantity.

3.4.2 Penalised Linear Regression

Given the feature matrix $X \in \mathbb{R}^{n \times p}$ built in Section 3.3.3 and the target vector y, ordinary least squares is unstable when predictors are highly correlated, as lagged and rolling-window features of the same series necessarily are. Ridge regression [Hoerl and Kennard, 1970] controls this by penalising the size of the coefficient vector:

$$\hat{\beta} \;=\; \arg\min_{\beta} \; \|y - X\beta\|_2^2 \;+\; \lambda \|\beta\|_2^2, \tag{3.9}$$

Unlike every other regressor in this section, Equation 3.9 has a closed-form minimiser rather than one found by an iterative search. Writing the objective as $J(\beta) = (y - X\beta)^\top (y - X\beta) +$

$\lambda\beta^\top\beta$ and differentiating with respect to β,

$$\nabla_\beta J \;=\; -2X^\top(y - X\beta) \;+\; 2\lambda\beta, \tag{3.10}$$

setting Equation 3.10 to zero gives the penalised normal equations, $(X^\top X + \lambda I)\hat{\beta} = X^\top y$, so

$$\hat{\beta} \;=\; (X^\top X + \lambda I)^{-1} X^\top y. \tag{3.11}$$

$X^\top X + \lambda I$ is positive definite for any $\lambda > 0$ even when $X^\top X$ itself is singular or near singular, which is exactly the collinear case the opening sentence of this subsection describes, so Equation 3.11 exists and is unique where the ordinary least squares solution, $\lambda = 0$, may not. The penalty $\lambda > 0$ shrinks the coefficients toward zero, trading a small increase in bias for a reduction in variance; λ is selected by cross-validation on the training block at every refit.

3.4.3 Random Forest

A random forest [Breiman, 2001] fits B regression trees to independent bootstrap resamples of the training data, each tree considering only a random subset of features at every split, and averages their predictions. Averaging over bootstrap resamples in this way is bootstrap aggregating, or bagging [Breiman, 1996]; a random forest adds the random feature subset at each split on top of bagging specifically to decorrelate the resulting trees further, since trees grown on overlapping bootstrap samples of the same series would otherwise tend to split on the same dominant lag features and average out less variance than independent trees would:

$$\hat{f}(x) \;=\; \frac{1}{B}\sum_{b=1}^{B} T_b(x). \tag{3.12}$$

Why decorrelation matters beyond bagging alone follows from the variance of an average of B identically distributed but not independent predictors, each with variance σ^2 and pairwise correlation ρ,

$$\mathrm{Var}\big(\hat{f}(x)\big) \;=\; \rho\sigma^2 \;+\; \frac{1-\rho}{B}\sigma^2, \tag{3.13}$$

the bound Breiman [2001] derives as the reason a random forest can outperform plain bagging rather than merely matching it. The second term in Equation 3.13 vanishes as B grows, but the first does not, so $\rho\sigma^2$ is a floor that averaging alone, however many trees, cannot fall below. Bagging without the random feature subset still has every tree free to split on whichever lag dominates, which keeps ρ high; restricting each split to a random subset of features is specifically what lowers ρ, and with it the floor itself, a mechanism unrelated to how many trees are averaged.

Each tree T_b is grown by recursively partitioning the feature space: at a node with data

$\{(x_i, y_i)\}$, the split (j, s) on feature j at threshold s is chosen to minimise the total within-region sum of squared error,

$$\min_{j,s} \left[\sum_{i : x_{ij} \le s} (y_i - \bar{y}_{R_1})^2 + \sum_{i : x_{ij} > s} (y_i - \bar{y}_{R_2})^2 \right], \tag{3.14}$$

where R_1 and R_2 are the two resulting regions and $\bar{y}_{R_1}$, $\bar{y}_{R_2}$ their mean targets, the splitting rule set out for classification and regression trees generally and collected in Breiman et al. [2017]. Averaging over many such trees, each trained on a different resample and a different random subset of features at each split, reduces the variance of a single tree's prediction without imposing a linear functional form.

3.4.4 Histogram Gradient Boosting

Gradient boosting [Friedman, 2001] fits an ensemble additively rather than by averaging independent trees, generalising an older idea, boosting, in which many weak learners are combined into one strong learner by repeatedly reweighting the examples a classifier got wrong so that the next learner in the sequence focuses on them [Freund and Schapire, 1997]. Friedman's reformulation recasts that reweighting as gradient descent in function space rather than in the space of example weights, which is what lets the same framework cover arbitrary differentiable loss functions rather than only classification error. The justification is a first-order Taylor expansion of the loss around the current ensemble F_{m-1}, treating the update h added at that point as the variable being optimised,

$$L\big(y_i, F_{m-1}(x_i) + h(x_i)\big) \approx L\big(y_i, F_{m-1}(x_i)\big) + h(x_i) \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F = F_{m-1}}, \tag{3.15}$$

which, to this linear order, is decreased fastest for a step of fixed size by setting $h(x_i)$ in the direction opposite the bracketed gradient at every training point: an infinite-dimensional analogue of ordinary gradient descent, with one coordinate per training example rather than one per model parameter, which is the sense in which this is descent in function space rather than in the space of example weights. Fitting the gradient exactly at every training point would reproduce the training data and generalise nowhere, so the update is instead constrained to be a regression tree, fitted not to the gradient itself but to its value at the training points, the pseudo-residual

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F = F_{m-1}}, \tag{3.16}$$

The ensemble is initialised as a constant, $F_0(x) = \arg\min_\gamma \sum_i L(y_i, \gamma)$, and for $m = 1, \ldots, M$ a regression tree h_m is fitted to this target, which for squared-error loss reduces to the ordinary

residual $r_{im} = y_i - F_{m-1}(x_i)$, and the ensemble is updated by

$$F_m(x) \;=\; F_{m-1}(x) \;+\; \nu\, h_m(x), \tag{3.17}$$

where $0 < \nu \leq 1$ is a shrinkage (learning rate) parameter that controls how much each new tree is allowed to correct the ensemble. The histogram-based implementation used here bins continuous features before searching for splits, which accepts missing values natively, consistent with the decision in Section 3.3.3 never to impute. This is the same binning strategy that made gradient boosting practical on large tabular datasets in implementations such as XGBOOST [Chen and Guestrin, 2016]; scikit-learn's histogram-based estimator used here applies the same idea at the scale this project needs, a few thousand rows rather than millions, so the speed advantage that motivated the original design matters less here than the fact that it accepts missing values without a separate imputation step. Both the number of boosting iterations M and the per-tree leaf count were chosen at every refit from a small explicit grid, since an uncontrolled M overfits badly on a series of this length, a failure mode encountered and corrected during development and reported in Section 5.3.

Both the random forest and the gradient boosting models were fitted twice, once to the level of log concentration and once to its change from the forecast origin, since a tree cannot extrapolate beyond the range of targets it was trained on and this series drifts across the decade of record; modelling the change removes that handicap by asking the tree to represent a roughly stationary residual rather than a trending level.

3.5 Deep Learning Methods

3.5.1 The Gated Recurrent Unit: Architecture and Training

The gated recurrent unit [Cho et al., 2014] maintains a hidden state $h_t \in \mathbb{R}^H$ updated at each timestep t by two gates. The update gate z_t controls how much of the previous state is retained against how much is replaced by a new candidate state; the reset gate r_t controls how much of the previous state is used in forming that candidate:

$$z_t = \sigma\big(W_z x_t + U_z h_{t-1} + b_z\big), \tag{3.18}$$

$$r_t = \sigma\big(W_r x_t + U_r h_{t-1} + b_r\big), \tag{3.19}$$

$$\tilde{h}_t = \tanh\big(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h\big), \tag{3.20}$$

$$h_t = (1 - z_t) \odot h_{t-1} \;+\; z_t \odot \tilde{h}_t, \tag{3.21}$$

where σ is the logistic sigmoid, $\odot$ is elementwise multiplication, and W_z, W_r, W_h, U_z, U_r, U_h, b_z, b_r, b_h are the parameters learned by training, fitted by backpropagation through time, the

sequential application of the backpropagation algorithm [Rumelhart et al., 1986] across every timestep in the input window. Because the final state h_t in Equation 3.21 is a convex combination of the previous state and the new candidate rather than a product of many successive nonlinear transformations, gradients can flow backward through many timesteps without vanishing in the sense formalised by Bengio et al. [1994], which is the property that motivated gated architectures in Section 2.3.

The specific network fitted in this project is a single GRU layer with $H = 24$ hidden units, reading a sequence of the last 30 days ($t-29, \ldots, t$), each day carrying six channels: the logarithm of concentration, a missingness flag, and two pairs of annual Fourier terms ($K = 2$), a smaller harmonic set than the $K = 4$ used for the ARIMA specification in Section 3.4.1, inherited from an earlier draft of the architecture rather than a deliberately tested choice. Section 5.3 reports a direct check of $K = 4$ for the network as well, which improves it at every horizon and substantially so at three days ahead, so the smaller set is better read as an unexamined default than a justified one. A linear output layer maps the final hidden state h_t to a single predicted value, the change in log concentration from the forecast origin, with the origin value added back afterwards; modelling the change rather than the level is the same device used for the tree models in Section 3.4, for the same reason. The network is trained by minimising mean absolute error with the Adam optimiser, with early stopping on a chronological holdout, the most recent 15 per cent of each training window, so that training stops once validation error no longer improves rather than at a fixed epoch count.

Writing d for the number of input channels, Equations 3.18 to 3.21 fix weight matrices $W_z, W_r, W_h \in \mathbb{R}^{H \times d}$ and $U_z, U_r, U_h \in \mathbb{R}^{H \times H}$ for the three gates, $3H(d+H)$ weights before a single bias is counted. The implementation used here, PyTorch's `nn.GRU`, trains each gate's bias as two separately learned halves rather than the one term b_z, b_r, b_h that Equations 3.18 to 3.21 show, one added to the input term and one to the hidden term and the two summed inside the gate, a convention shared with cuDNN-backed implementations generally rather than a departure from what those equations state. That doubles the bias count the equations alone would suggest, $6H$ rather than $3H$, giving

$$3H(d+H+2) \; + \; (H+1) \tag{3.22}$$

trainable parameters in total, the second term the linear output layer's own weight vector and bias. For $H = 24$, Equation 3.22 gives 2,329 parameters for the six-channel network, $d = 6$, and 2,617 for the ten-channel weather version, $d = 10$, both confirmed directly by counting the fitted model objects' own parameters rather than trusted as arithmetic.

A recurrent network forces a compromise that no other model in this project requires. It must be handed a value at every timestep: it cannot skip one the way the state space model in Section 3.4.1 skips a measurement update, and it cannot accept a missing value the way the

histogram gradient boosting model in Section 3.4 can. Gaps are therefore carried forward in the log-concentration channel, with the separate missingness flag marking which days were filled rather than observed, the same forward-fill-plus-indicator construction Che et al. [2018] use for recurrent networks over clinical time series with the same structural gap, a value required at every timestep and a sensor that does not always provide one. The one architecture Dr Tian specifically asked to be included is the one architecture in the suite that cannot represent the statement that a value is unknown.

A second version of this network reads two further channels, the day's mean temperature and wind speed from Section 3.3.4, each paired with its own missingness flag in exactly the same construction used for the concentration channel, ten channels in total rather than six. Wind speed in particular is a plausible carrier of the short range structure that already let the plain network beat ARIMA at one day ahead in the negative result anticipated in Section 2.3: a calm, stagnant airmass is both the immediate physical precondition for a particulate episode and a quantity the Fourier terms, which know only the calendar, cannot represent at all. Results for both versions are given in Section 4.1.

Because the network's weights are initialised randomly, its forecast depends on that initialisation, unlike every other model in this project. It is therefore fitted independently across ten random seeds, and its result is reported throughout Chapter 4 as the range across those ten runs, never as a single seed's point estimate; the justification for treating this as a reporting requirement rather than a presentational choice is given in Section 4.1. The sample was doubled from an initial five seeds to ten after rerunning the full pipeline on this same machine reproduced every classical and tree-based model's score exactly but moved the network's by nearly as much as the gap between two genuinely different seeds, evidence that its training is not bit-reproducible even with `-seed` fixed; Section 5.3 returns to what that means for the results that follow. The architecture itself, $H = 24$ and a 30-day window, was fixed by inspection rather than by the held-out search used for the tree models' capacity in Section 3.4; Section 5.3 also reports a bounded check of that choice, run after the fact.

3.5.2 The Long Short-Term Memory Network: Architecture and Training

Dr Tian's own review of a draft of this chapter suggested testing the GRU-over-LSTM choice directly rather than leaving it to the parameter-count argument alone (Section 2.3). This section does so. The long short-term memory network [Hochreiter and Schmidhuber, 1997] maintains both a hidden state $h_t \in \mathbb{R}^H$ and a separate cell state $c_t \in \mathbb{R}^H$, updated at each timestep by three gates rather than the GRU's two. The forget gate f_t controls how much of the previous cell state is retained; the input gate i_t controls how much of a new candidate is written to it; the output

gate o_t controls how much of the (squashed) cell state is exposed as the hidden state:

$$f_t = \sigma\big(W_f x_t + U_f h_{t-1} + b_f\big), \tag{3.23}$$

$$i_t = \sigma\big(W_i x_t + U_i h_{t-1} + b_i\big), \tag{3.24}$$

$$o_t = \sigma\big(W_o x_t + U_o h_{t-1} + b_o\big), \tag{3.25}$$

$$\tilde{c}_t = \tanh\big(W_c x_t + U_c h_{t-1} + b_c\big), \tag{3.26}$$

$$c_t = f_t \odot c_{t-1} \; + \; i_t \odot \tilde{c}_t, \tag{3.27}$$

$$h_t = o_t \odot \tanh(c_t), \tag{3.28}$$

with σ, $\odot$, and every weight and bias playing the same role as in the GRU equations above, now across four gates instead of two. The explicit cell state c_t, absent from the GRU, is the LSTM's distinguishing mechanism: the forget and input gates let it carry information additively across timesteps largely undisturbed, rather than always being reconstructed as a convex combination of the previous and candidate state the way the GRU's h_t is in Equation 3.21.

The network fitted here is identical in every respect to the GRU of Section 3.5.1 except the recurrent cell itself: the same six channels (log concentration, missingness flag, two annual Fourier pairs), the same 30-day window, the same $H = 24$ hidden units, the same change-in-log target with the origin value added back, the same Adam optimiser minimising mean absolute error with early stopping on the same chronological holdout, and the same treatment across ten random seeds, reported as a range rather than a point for the same reason. Holding every other design choice fixed isolates the one comparison this subsection exists to make: GRU against LSTM, not GRU against a differently-tuned LSTM.

Following the same parameter count convention as Equation 3.22 (PyTorch's `nn.LSTM` splits each gate's bias into two separately learned halves, the same departure from Equations 3.23 to 3.28 noted for the GRU), four gates rather than three give

$$4H(d + H + 2) \; + \; (H + 1) \tag{3.29}$$

trainable parameters. For $H = 24$ and the six-channel network, $d = 6$, Equation 3.29 gives 3,097 parameters, confirmed directly by counting the fitted model object's own parameters, roughly a third more than the GRU's 2,329 for the same channels. Results are given alongside the GRU's in Section 4.1; the weather-augmented channel set used for the GRU in Section 3.3.4 was not refitted for the LSTM, nor was the decision-threshold sweep of Section 4.3 rerun with it, a scope boundary stated explicitly in Section 5.3 rather than left for a reader to notice on their own.

3.5.3 The Transformer: Architecture and Training

Dr Tian's second review raised the transformer conditionally: try it only "if the size is large." Section 2.3.1 judged that this project's roughly three thousand daily observations do not meet that condition, and the GRU was preferred on that basis. His reply to a bounded, single-split check of that argument (Section 5.3) was direct: counting every evaluation block the series is some four thousand days long, which by his own reading of the literature is well above the few hundred observations needed before a transformer is worth trying, and the small-data case should be tested properly rather than left to stand on a bounded check alone. This section gives the architecture that test fits; the full ten-seed rolling-origin result is in Section 4.1.

A transformer encoder [Lim and Zohren, 2021] replaces recurrence with self-attention: rather than propagating a hidden state step by step the way the GRU and LSTM above do, every timestep in the input window attends directly to every other one. Each input vector $x_t \in \mathbb{R}^d$ in the 30-day window is first linearly projected to a d_{model}-dimensional embedding, $e_t = W_e x_t + b_e$, deliberately distinct notation from the hidden size H used for the GRU and LSTM above since this is an embedding width, not a recurrent state. Self-attention has no inherent sense of position, unlike a recurrent cell whose state is built up one timestep at a time, so a fixed sinusoidal positional encoding is added elementwise before the first layer,

$$PE_{(pos,2i)} = \sin\Big(pos \,/\, 10000^{2i/d_{model}}\Big), \tag{3.30}$$

$$PE_{(pos,2i+1)} = \cos\Big(pos \,/\, 10000^{2i/d_{model}}\Big), \tag{3.31}$$

for position $pos \in \{0,\dots,29\}$ and embedding index $2i < d_{model}$, the construction Lim and Zohren [2021] describe for attention-based forecasters generally, giving $z_t = e_t + PE_t$.

Writing $Z \in \mathbb{R}^{30\times d_{model}}$ for the whole window stacked as a matrix, three learned projections give queries, keys and values, $Q = ZW^Q$, $K = ZW^K$, $V = ZW^V$ with $W^Q, W^K, W^V \in \mathbb{R}^{d_{model}\times d_{model}}$, and attention output is a weighted average of the values, the weight on each position set by how well its key matches every query,

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_{model}}}\right)V, \tag{3.32}$$

the scaling by $\sqrt{d_{model}}$ keeping the softmax's argument from growing with the embedding width and pushing gradients toward saturation. Multi-head attention runs n_{head} independent copies of Equation 3.32 on d_{model}/n_{head}-dimensional subspaces of Q, K, V in parallel, concatenates their outputs, and projects the result back to d_{model} dimensions with one further learned matrix, letting the model represent more than one kind of relationship between positions at once rather than averaging them into a single attention pattern.

Each encoder layer wraps this attention sublayer, and a position-wise feed-forward network

applied identically to every position, in a residual connection and layer normalisation around each:

$$Z' = \mathrm{LayerNorm}\big(Z + \mathrm{MultiHead}(Z)\big), \tag{3.33}$$

$$Z'' = \mathrm{LayerNorm}\big(Z' + W_2\,\mathrm{ReLU}(W_1 Z' + b_1) + b_2\big), \tag{3.34}$$

with $W_1 \in \mathbb{R}^{d_{ff} \times d_{model}}$, $W_2 \in \mathbb{R}^{d_{model} \times d_{ff}}$ expanding to a feed-forward width d_{ff} and projecting back; each sublayer also applies dropout during training, PyTorch's default rate of 0.1, left unchanged and omitted from Equations 3.33 and 3.34 since it is a regulariser rather than part of the architecture's input-output map at inference. A linear output layer maps the final position's representation, row 30 of Z'', to a single predicted value, the change in log concentration from the forecast origin, the same convention Equations 3.21 and 3.28 use for the recurrent networks' final hidden state.

The network fitted here is otherwise identical to the GRU and LSTM above: the same six channels (log concentration, missingness flag, two annual Fourier pairs), the same 30-day window, the same change-in-log target with the origin value added back, the same Adam optimiser with early stopping on the same chronological holdout, and the same treatment across ten random seeds, reported as a range for the same reason. Four settings are specific to this architecture and were not independently tuned: $d_{model} = 16$, $n_{head} = 2$ attention heads, a single encoder layer with feed-forward width $d_{ff} = 4d_{model} = 64$, and an Adam learning rate of 0.001 rather than the 0.01 used for the GRU and LSTM, all inherited unchanged from the single-split bounded check (Section 5.3) that preceded the full evaluation. Section 5.3 reports bounded sensitivity checks for the GRU's own hyperparameters; no equivalent check exists yet for the transformer's, which is stated here rather than left for a reader of Table 4.1 to assume.

Summing the input projection, one encoder layer's self-attention and feed-forward sublayers with their two layer normalisations, and the output layer,

$$d_{model}\big(12\,d_{model} + d + 15\big) \;+\; 1 \tag{3.35}$$

trainable parameters in total, for a single encoder layer. For $d_{model} = 16$ and the six-channel network, $d = 6$, Equation 3.35 gives 3,409 parameters, confirmed directly by counting the fitted model object's own parameters: the largest of the three deep learning architectures in this project, ahead of the LSTM's 3,097 and the GRU's 2,329, and independent of n_{head}, which only reshapes the existing query, key and value projections rather than adding new ones.

3.6 Rolling-Origin Evaluation

Every model was evaluated over 2,303 origins from 9 November 2019 to 24 August 2026, spanning 6.8 years, with a three year burn-in and model state advanced daily between refits. Parameters are re-estimated every 90 days for every specification except the three networks, the GRU, the LSTM and the transformer, which refit every 180 days: each of their refits trains a network from scratch rather than solving a closed form or appending one more observation to an already-converged filter, so halving the refit frequency is the practical cost of including a trained network in the suite at all rather than an oversight.

This is not a refinement of a single train-test split; it replaces one, for the reason standard cross-validation is inappropriate for dependent data in the first place: shuffling origins lets a model trained on the future score itself against the past [Bergmeir and Benítez, 2012]. On a single test block the ARIMA scored 0.903 in relative terms at one day ahead; under the rolling-origin harness built to replace it, the same model scores 0.940, worse rather than better, because the single block happened to be calmer than the training period and was flattering every model. A single split is one draw from a process, and a conclusion drawn from it is a conclusion about that period, exactly the instability in single-split evaluation that Tashman [2000] documents across decades of applied forecasting studies and that Hewamalage et al. [2023] report as still common practice. The rolling-origin design used throughout this project is adopted specifically to avoid being another instance of the pattern both papers describe.

The governing invariant is that at origin t a model may use observations up to and including day t and nothing else: not the value at $t+1$, not a parameter estimated using data after t, and not a scaling constant or imputed value computed from the full series. Since nothing fails visibly when that is broken, and the numbers merely improve, the invariant is enforced by construction and tested directly. Every model runs through one shared loop, and that loop is verified by corrupting the series from a chosen date, rerunning, and requiring that no forecast issued before that date changes.

One trap is specific to supervised learning under a rolling origin. At origin t the label attached to feature row t' is the concentration at $t'+h$, so the training set ends at $t-h$ rather than at t. A separate model is fitted per horizon because that arithmetic requires it. The error would have amounted to three rows per refit and would have been entirely invisible in the output.

3.7 Evaluation Measures

Accuracy is reported separately at each horizon and never averaged, since skill decays with horizon and concealing that would defeat the purpose. Writing $e_i = \hat{y}_i - y_i$ for the forecast error of row i among the n rows a model is scored on, the headline measure is relative mean absolute

error against the persistence forecast, computed on identical rows,

$$\text{rMAE} \;=\; \frac{\text{MAE(model)}}{\text{MAE(persistence)}}, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \tag{3.36}$$

so that 1.000 is persistence and values below one beat it. Mean absolute scaled error,

$$\text{MASE} \;=\; \frac{\text{MAE(model)}}{\frac{1}{n_{\text{train}}-1}\sum_{t=2}^{n_{\text{train}}} |y_t - y_{t-1}|}, \tag{3.37}$$

proposed by Hyndman and Koehler [2006] as a scale-free accuracy measure that remains defined even when the series passes through values near zero, is reported alongside, but its denominator is fixed on the training period and therefore also reflects any difference in volatility between training and test periods, which is material here; this is the same distinction between a scaled error and a relative error that motivated reporting both rather than either alone.

As a warning system, a day is an event if the observed concentration exceeds the health threshold, and an alarm is raised when the forecast exceeds a decision threshold. These are separate quantities and are kept separate throughout. Every warning-system measure reported is a ratio of four counts accumulated over the days a model is scored on: true positives TP, an alarm on an event day; false positives FP, an alarm on a quiet day; false negatives FN, a missed event; and true negatives TN, a quiet day correctly left unflagged. Reported are the hit rate,

$$\text{hit rate} \;=\; \frac{\text{TP}}{\text{TP}+\text{FN}}, \tag{3.38}$$

the share of actual events the model catches; the false alarm rate, defined as false alarms divided by quiet days rather than by alarms raised,

$$\text{false alarm rate} \;=\; \frac{\text{FP}}{\text{FP}+\text{TN}}; \tag{3.39}$$

precision, the share of alarms that were right,

$$\text{precision} \;=\; \frac{\text{TP}}{\text{TP}+\text{FP}}; \tag{3.40}$$

the critical success index [Schaefer, 1990],

$$\text{CSI} \;=\; \frac{\text{TP}}{\text{TP}+\text{FP}+\text{FN}}, \tag{3.41}$$

which discards TN entirely; and alarms raised per year, $365.25\,(\text{TP}+\text{FP})/n$, included because a false alarm rate of 0.10 sounds negligible and means roughly one wasted alarm every ten quiet days, which is what an operator actually experiences. Plain classification accuracy, $(\text{TP}+\text{TN})/n$, is deliberately never reported as a warning-system measure: with hazardous days a

minority class throughout the record, TN dominates n, so a model that never raises an alarm, $\mathrm{TP} = \mathrm{FP} = 0$, scores TN/n on accuracy, high by construction, while catching none of the events it exists to catch [He and Garcia, 2009]. Equations 3.38 to 3.41 are exactly the decomposition that distortion motivates, a precision-recall-style reading of the same contingency table rather than a single aggregate rate on imbalanced data [Saito and Rehmsmeier, 2015].

3.8 Statistical Comparison of Forecasts

Section 5.2 bounds how large a gap between two models' scores must be before it is more than measurement noise, but that bound is empirical: it is the amount a score moved when nothing about the model or the data changed, not a sampling distribution with a p-value attached. The Diebold-Mariano test [Diebold and Mariano, 1995] supplies the formal version of the same question. For two models with forecast errors $e_{1,t}$ and $e_{2,t}$ at the same horizon h and the same forecast origins, define the loss differential

$$d_t \;=\; L(e_{1,t}) \;-\; L(e_{2,t}), \tag{3.42}$$

with $L(e) = |e|$ throughout, matching this report's own headline measure, Equation 3.36, rather than squared error. Under the null hypothesis of equal predictive accuracy, $E[d_t] = 0$, and the test statistic is

$$\mathrm{DM} \;=\; \frac{\bar{d}}{\sqrt{\widehat{\mathrm{Var}}(\bar{d})}}, \qquad \bar{d} = \frac{1}{n}\sum_{t=1}^{n} d_t, \tag{3.43}$$

asymptotically standard normal under the null. The variance estimate is not the plain sample variance divided by n: an optimally specified h-step forecast has errors that are at most an $\mathrm{MA}(h-1)$ process, so the loss differential it produces carries genuine serial correlation up to lag $h-1$, which Diebold and Mariano's derivation folds into a long-run variance,

$$\widehat{\mathrm{Var}}(\bar{d}) \;=\; \frac{1}{n}\left[\hat{\gamma}_0 \;+\; 2\sum_{k=1}^{h-1}\hat{\gamma}_k\right], \qquad \hat{\gamma}_k \;=\; \frac{1}{n}\sum_{t=k+1}^{n}(d_t-\bar{d})(d_{t-k}-\bar{d}), \tag{3.44}$$

rather than the plain estimator a reader more familiar with an ordinary paired t-test might expect; at $h = 1$ there is nothing to correct for and Equation 3.44 reduces to the ordinary case, $\hat{\gamma}_0/n$. Because this project's longest horizon is three days, the correction is never more than two autocovariance terms. The asymptotic normal approximation can be anti-conservative at the sample sizes forecasting studies actually have, so this report uses the small-sample correction of Harvey et al. [1997],

$$\mathrm{DM}^{*} \;=\; \mathrm{DM}\left[\frac{n+1-2h+h(h-1)/n}{n}\right]^{1/2}, \tag{3.45}$$

referred to a t_{n-1} distribution rather than the standard normal, which is uniformly more conservative and is what every statistic and p-value attributed to this test in Chapter 4 actually uses. That correction addresses the sample size directly, but the asymptotic justification behind both the uncorrected and the corrected statistic still rests on a central limit theorem applying to $\bar{d}$, and this report has shown PM2.5 itself to be strongly right skewed with occasional extreme episodes: a loss differential built from absolute errors on that scale can inherit some of the same heavy-tailedness, which is exactly the condition under which a normal approximation converges more slowly. Nothing in Chapter 4 checks this directly, for instance by bootstrapping the statistic's own sampling distribution rather than relying on the asymptotic one; the correction applied here addresses a known and specific failure mode of the test, not every way its normal approximation could be imprecise on a series with this one's particular shape.

This is applied to exactly the comparison Section 4.1 already makes by counting seed signs against the empirical noise floor: the recurrent network against the ARIMA benchmark, with and without weather, at every horizon. Because the network alone depends on a random initialisation, the test is run once per seed against the same ARIMA forecast and reported as the range across all ten, exactly the convention already used for every other seed-dependent number in this report, rather than as a single run's p-value that a different seed could have produced differently.

3.9 Reproducibility

The analysis runs end to end from an empty directory via twelve numbered pipeline scripts, the transformer of Section 3.5.3 the most recent addition, eleven of which carry an automated correctness test, run with `-test`, that exits before producing any output if its own checks fail. The rolling origin harness verifies its own leakage invariant before every run. Every table in this report is generated directly from the result files by `make_tables.py`; no number has been transcribed by hand. Raw and cleaned data are held under version control so that the analysis does not depend on the data provider serving identical values at a later date, which the finding in Section 3.2 suggests is not a safe assumption. The complete repository, including every commit from initial data acquisition to this report, is public and runnable at `github.com/JC16092/mth5000-aqi-forecast`.

3.10 Chapter Summary

This chapter described the data, two defects found in it, and the feature table built from it, including an optional weather extension (Section 3.3.4) carrying its own third data-quality defect, a hard cutoff rather than scattered gaps, and then gave the full mathematical specification of

the two method families compared in this project. Machine learning, in Section 3.4, comprises a classical ARIMA benchmark and three regressors, ridge, random forest, and gradient boosting, each controlling the bias-variance trade-off by a different mechanism: shrinkage, bagging, or additive stagewise correction. Deep learning, in Section 3.5, comprises three networks: a GRU and an LSTM fitted under an otherwise identical protocol specifically to test the choice between them directly, each avoiding the vanishing-gradient failure of a plain recurrent network through a different gating mechanism, two gates and no separate memory cell for the GRU, three gates and an explicit cell state for the LSTM, and a transformer (Section 3.5.3) that replaces recurrence with self-attention altogether, added once Dr Tian's own feedback questioned whether this project's data was really too small for one. Every model is evaluated under the same rolling-origin protocol, specified in Section 3.6, and scored by the same measures, specified in Section 3.7, so that the comparison in Chapter 4 is a fair one; where that comparison involves a network against the ARIMA benchmark, it is also given a formal significance test, specified in Section 3.8.

Chapter 4

Results and Analysis

4.1 Forecast Accuracy

Table 4.1: Relative mean absolute error against persistence, under rolling origin evaluation over 2,303 origins. Lower is better and 1.000 is persistence. The three networks, the GRU, the LSTM and the transformer, are each given as the range over ten initialisation seeds; they are the only models whose result depends on one.

Model	$h = 1$	$h = 2$	$h = 3$	Bias at $h = 1$
GRU, 30 day sequence, range over 10 seeds	0.910 to 0.939	0.878 to 0.906	0.831 to 0.879	-2.8
LSTM, 30 day sequence, range over 10 seeds	0.913 to 0.932	0.874 to 0.892	0.824 to 0.847	-2.4
Transformer, 30 day sequence, range over 10 seeds	0.914 to 0.925	0.870 to 0.893	0.825 to 0.842	-2.8
ARIMA(1,1,1) with Fourier terms	0.940	0.883	0.832	-6.4
Random forest, change target	0.968	0.919	0.897	+2.1
Gradient boosting, change target	0.983	0.937	0.878	+2.0
Random forest, level target	0.988	0.934	0.879	+4.1
Gradient boosting, level target	1.008	0.945	0.885	+4.0
Ridge regression	0.997	0.952	0.897	+2.6
Persistence	1.000	1.000	1.000	-0.1
Climatology	1.415	1.060	0.947	+9.4
Seasonal naive, lag 7	1.759	1.325	1.168	+0.1

Table 4.1 and Figure 4.1 answer the first research question, and the answer is not uniform across horizons.

At one day ahead the GRU is the best model, and this is robust: across ten initialisation seeds it is numerically ahead of the ARIMA specification in every one, and clearly so, beyond this report's own 0.004 noise floor (Section 5.2), in eight of the ten; the two weakest, 0.936 and 0.939 against ARIMA's 0.940, are close enough to call ties rather than clear wins. At two days the picture is closer: ARIMA is numerically ahead in nine of the ten seeds, but only five of those nine clear the noise floor, one seed (0.878) beats ARIMA outright, and the remaining four are indistinguishable from it. At three days neither model has a seed that clears the floor against the other: the best network seed, 0.831, and ARIMA's 0.832 are separated by a few parts in ten thousand, a tie in any language that would survive examination, while the worst network seed,

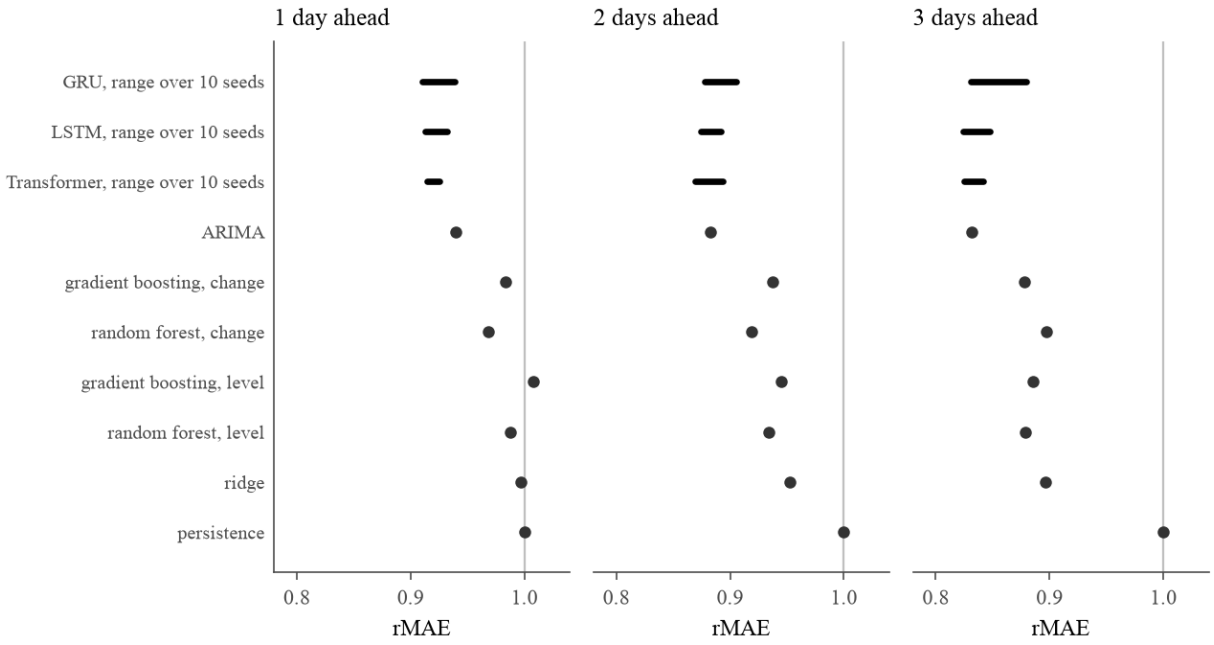


Figure 4.1: Table 4.1 as a dot plot, one row per model, one panel per horizon, the vertical line at 1.000 marking persistence. None of the three seeded networks has a single dot: each one's ten seeds are shown as a horizontal range bar rather than a point estimate. Climatology and the seasonal-naive benchmark are omitted here, both well outside this axis range at one day ahead; their numbers are unchanged in Table 4.1.

0.879, is clearly the weaker of the two.

Table 4.2 puts a formal test behind the comparison the paragraph above makes by counting seed signs against an empirical floor. At one day ahead the statistic favours the GRU in all ten seeds and the advantage reaches $p < 0.05$ in four of them. At two days, every seed's p-value exceeds 0.07 and none reaches significance. At three days exactly one seed does, and in ARIMA's favour, $p = 0.0056$; the rest do not. Section 5.2 returns to this table directly, since it asks the same question the empirical noise floor does and answers it more conservatively at every horizon reported there.

The mechanism is only partly architectural. At one day ahead the forecast is dominated by short-range structure, where a nonlinear model has something to exploit. As the horizon lengthens the seasonal component takes over, and ARIMA's four Fourier pairs carry more of it than the network's two (Section 3.5.1), not because the network lacks an explicit seasonal signal altogether but because it receives a coarser one. Section 5.3 reports a bounded check of exactly that difference, and it accounts for a meaningful share of the gap at three days.

The GRU is reported as a range rather than a point because, until the LSTM below, it was the only model whose result depends on a random initialisation, and the dependence is not small. Across ten seeds the spread is 0.029 at one day, 0.028 at two and 0.048 at three. At three days that spread is wider than the gap separating the ARIMA from two of the other models in the table, so a single seed could have been reported as beating them or losing to them at the

Table 4.2: Diebold-Mariano test of each network against the ARIMA benchmark (Section 3.8), run once per seed and given as the range across all ten. A negative statistic favours the network at that seed; a positive one favours ARIMA. The last column counts seeds reaching $p < 0.05$, two-sided.

Comparison	h	DM statistic	p-value	Sig. seeds
Plain GRU vs. ARIMA	1	-3.08 to -0.10	0.002 to 0.923	4/10
Plain GRU vs. ARIMA	2	-0.47 to 1.73	0.083 to 0.992	0/10
Plain GRU vs. ARIMA	3	-0.05 to 2.77	0.006 to 0.987	1/10
GRU with weather vs. ARIMA	1	-3.92 to -0.87	<0.001 to 0.383	7/10
GRU with weather vs. ARIMA	2	-1.78 to 0.26	0.075 to 0.942	0/10
GRU with weather vs. ARIMA	3	-0.83 to 1.97	0.049 to 0.907	1/10
LSTM vs. ARIMA	1	-2.59 to -0.66	0.010 to 0.511	5/10
LSTM vs. ARIMA	2	-0.65 to 0.83	0.409 to 0.974	0/10
LSTM vs. ARIMA	3	-0.59 to 1.05	0.292 to 0.995	0/10
Transformer vs. ARIMA	1	-2.74 to -1.39	0.006 to 0.164	7/10
Transformer vs. ARIMA	2	-1.34 to 0.84	0.181 to 0.995	0/10
Transformer vs. ARIMA	3	-0.63 to 0.73	0.463 to 0.988	0/10

analyst's convenience.

The LSTM, fitted on an identical protocol (Section 3.5.2). Table 4.1 shows it ahead of the GRU at every horizon, by 0.005 at one day, 0.008 at two and 0.013 at three, despite carrying roughly a third more parameters, 3,097 against 2,329. That directly tests, and does not support, the parameter-count reasoning Section 2.3 gives for preferring the smaller architecture: on this evidence the smaller network is not winning because it is smaller. Against ARIMA specifically, the LSTM's own picture mirrors the GRU's more than it departs from it. At one day ahead it is numerically ahead in all ten seeds (mean rMAE 0.919 against ARIMA's 0.940) and Table 4.2 reaches $p < 0.05$ in five of them, a formal result if anything slightly stronger than the GRU's own four of ten. At two days the two models are close to indistinguishable, 0.882 against ARIMA's 0.883, and no seed reaches significance either way. At three days ARIMA remains marginally ahead, 0.831 against the LSTM's 0.836, again with no seed reaching significance. The LSTM was not carried through the weather extension of Section 3.3.4 or the decision-threshold sweep of Section 4.3; Section 5.3 states this scope boundary directly rather than leaving Table 4.1 to imply a fuller treatment than the one actually given.

The transformer, promoted to the full treatment after Dr Tian's reply to the GRU-and-LSTM response (Section 3.5.3). Table 4.1 shows it ahead of the GRU in eight of the ten matched seed pairs at every horizon, and essentially matching rather than losing to the LSTM: ahead in five of ten at one day, seven of ten at two, and five of ten at three. Against ARIMA specifically, the transformer is numerically ahead in all ten seeds at one day ahead (mean rMAE 0.919 against ARIMA's 0.940, coincidentally the same mean the LSTM reaches above by a

different route) and Table 4.2 reaches $p < 0.05$ in seven of them, the strongest formal result of the three networks at this horizon, ahead of the LSTM's five and the GRU's four. At two days it is numerically ahead in eight of ten seeds, mean 0.879 against ARIMA's 0.883, but as with the GRU and LSTM no seed clears significance. At three days ARIMA remains ahead in most seeds, the transformer winning three of ten, mean 0.834 against ARIMA's 0.832, again with no seed on either side reaching significance. This result speaks directly to the small-data argument Section 2.3.1 makes against an attention-based architecture on a series this size: Dr Tian's own reading, that roughly four thousand days clears the threshold at which a transformer is worth trying, is closer to what this project's own evidence shows than the argument it was tested against. Like the LSTM, the transformer was not carried through the weather extension of Section 3.3.4 or the decision-threshold sweep of Section 4.3; Section 5.3 states this scope boundary directly for both networks together.

Two subsidiary results support the specification choices. Removing the Fourier terms from the ARIMA degrades it by 3.3 points of relative error at one day and by 5.8 points at three, under the same rolling-origin harness, so the seasonal terms earn their place and increasingly so with horizon. Modelling the change from the origin rather than the level improves both tree models, because a tree cannot extrapolate beyond the targets it was trained on and this series drifts downward across the decade.

Table 4.2 tests the four comparisons in this report with a seed to range over: the plain GRU, the weather-augmented GRU, the LSTM, and the transformer, each against ARIMA. The same test applies just as directly to the other five machine learning rows against ARIMA (ridge, and the random forest and gradient boosting's level and change-target variants): each is reported as a single fit rather than a range, so one Diebold-Mariano statistic per horizon is the whole comparison, not an average over several.

Table 4.3 backs the chapter's other central claim, that the classical benchmark beats every machine learning regressor at every horizon, with the same kind of formal test already given to the network comparison, rather than leaving it to point estimates and the informal noise floor alone. The statistic favours ARIMA in all fifteen model-horizon pairings, and fourteen reach $p < 0.05$; the one exception, the change-target random forest at two days, falls just short at $p = 0.054$. This is, if anything, a stronger formal result than the network comparison can supply for its own horizons, precisely because none of these five rows carries a seed's worth of sampling uncertainty on top of the test's own.

4.1.1 Does Weather Change the Answer?

The specification above uses no information beyond the series' own history. Section 3.3.4 adds daily wind speed and temperature, and the question is whether that changes which model wins.

Table 4.4 answers it unevenly across models, and the unevenness is itself informative. Gra-

Table 4.3: Diebold-Mariano test of ARIMA against each point-forecast machine learning regressor (Section 3.8). A negative statistic favours the machine learning model; a positive one favours ARIMA. An asterisk marks $p < 0.05$, two-sided.

Comparison	h	DM statistic	p-value
Ridge regression vs. ARIMA	1	4.25	$<0.001^*$
Ridge regression vs. ARIMA	2	3.85	$<0.001^*$
Ridge regression vs. ARIMA	3	3.62	$<0.001^*$
Random forest, level target vs. ARIMA	1	3.05	0.002^*
Random forest, level target vs. ARIMA	2	2.43	0.015^*
Random forest, level target vs. ARIMA	3	2.19	0.029^*
Random forest, change target vs. ARIMA	1	1.99	0.046^*
Random forest, change target vs. ARIMA	2	1.93	0.054
Random forest, change target vs. ARIMA	3	3.53	$<0.001^*$
Gradient boosting, level target vs. ARIMA	1	3.92	$<0.001^*$
Gradient boosting, level target vs. ARIMA	2	2.65	0.008^*
Gradient boosting, level target vs. ARIMA	3	2.26	0.024^*
Gradient boosting, change target vs. ARIMA	1	3.00	0.003^*
Gradient boosting, change target vs. ARIMA	2	2.51	0.012^*
Gradient boosting, change target vs. ARIMA	3	1.96	0.050^*

Table 4.4: Change in rMAE from adding weather covariates, same rolling-origin protocol and same reference as Table 4.1. Negative is an improvement. The network's weather row is given as its own absolute range over ten seeds, to be read against its row in Table 4.1.

Model	Δ rMAE, $h = 1$	Δ rMAE, $h = 2$	Δ rMAE, $h = 3$
Ridge regression	-0.031	-0.005	+0.023
Random forest, level target	+0.002	-0.006	+0.053
Random forest, change target	-0.003	-0.001	+0.022
Gradient boosting, level target	-0.082	-0.020	-0.029
Gradient boosting, change target	-0.073	-0.047	+0.006
GRU, 30 day sequence, with weather, 10 seeds (absolute)	0.882 to 0.926	0.855 to 0.887	0.816 to 0.873

dient boosting improves substantially and at every horizon, by 8.2 points of relative error at one day, 2.0 at two and 2.9 at three on the level target, and similarly on the change target. The random forest moves the other way at the longest horizon, 5.3 points worse at three days, while ridge improves at one and two days and worsens slightly at three. No single story explains all five machine learning rows; weather is not a free improvement for every method that can accept it.

The recurrent network is where the extension matters most, because Section 3.3.4 identified wind speed as a specific physical candidate for the short range structure the plain network was already exploiting at one day ahead. With weather added, the network beats ARIMA clearly under all ten seeds at one day ahead, as before. At two days, where ARIMA previously won outright, the picture becomes a genuine contest rather than a clean win for either side: three of ten seeds beat ARIMA clearly, beyond the 0.004 noise floor established in Section 5.2, six are indistinguishable from it within that floor, and one is a clear loss. At three days a similar three-way split appears with different seeds in each role: three of ten beat ARIMA clearly, two are ties, and five, a majority, are clear losses, including one seed 0.041 worse than ARIMA. Doubling the seed count from the five first fitted is what surfaced this; under the first five alone, both horizons had looked like a win for the network, and that reading did not survive a second, independent batch of five seeds.

The same formal test behind Table 4.2 reaches a similar verdict for the weather version, in its lower block. At one day ahead it favours the network in all ten seeds and reaches $p < 0.05$ in seven, a stronger formal result than the plain network's four, consistent with wind speed carrying genuine short-range information rather than noise that a larger input simply fits more closely. At two days the direction itself splits, six seeds favour the network and four favour ARIMA, and not one seed on either side reaches significance; by this reading the three-way split the seed-counting above reports is closer to no detectable difference at all than to a genuine three-way contest. At three days one seed reaches significance, $p = 0.049$, and it favours ARIMA, consistent with the majority-loss reading above, though a p-value that close to the threshold means a single further seed could move the count either way.

The honest summary is that wind speed and temperature measurably help the one architecture built to exploit short-range structure at one day ahead, leave the two-day comparison a genuine tie rather than the win it first appeared to be, and leave three days ahead more often a loss than a win for the network, while helping the tree-based machine learning models unevenly and the random forest not at all at the longest horizon. This was checked against the weather coverage gap diagnosed in Section 3.3.4 by repeating the comparison restricted to origins inside the fully observed weather window; the ordering at one day ahead survived that restriction, and so did the mixed, seed-dependent picture at two and three days, which is what makes the finding a property of the weather signal and of the network's own training instability, rather than of how the coverage gap was filled.

That accuracy gain was then carried through to Research Question 3 directly, by rerunning the threshold sweep from Section 4.3 with the weather models added to the roster of candidates at every budget, on exactly the same rows the accuracy comparison above used.

Table 4.5: Best achievable hit rate within an alarm budget, before and after adding the weather models as candidates, averaged over the five network seeds. The two rows at each horizon are scored on identical origins; the only difference is which models are allowed to be the best one.

Alarms per year	40	60	80	100
Best hit rate, $h = 1$, no weather	0.401	0.609	0.787	0.899
with weather models added	0.402	0.609	0.792	0.909
Best hit rate, $h = 2$, no weather	0.388	0.597	0.771	0.900
with weather models added	0.400	0.597	0.771	0.900
Best hit rate, $h = 3$, no weather	0.383	0.567	0.756	0.876
with weather models added	0.383	0.577	0.756	0.876

Table 4.5 gives the answer, and it favours the ceiling over the gain. At several budgets the weather models do not even supply the best option, delta exactly zero: 60 alarms a year at one day ahead; 60, 80 and 100 at two; and 40, 80 and 100 at three. Where they do win, the improvement is small: at most 1.2 points of hit rate, at two days and a budget of 40, and one point or under at every other budget where they help at all. That is well below the five-point spread across models documented in Table 4.7 and discussed in Section 5.2, not a new tier above it, and nowhere near the fifty-point range that moving the budget itself buys. A substantially more accurate point forecast, 8.2 points of relative error at one day ahead for gradient boosting, translates into at most a little over a point of additional hit rate once it is run through a decision threshold. The ceiling this report's central finding describes is not a property of the ten original models being insufficiently accurate; it persists even after a materially more accurate forecaster is added to the comparison.

4.2 Two Arms of the Same Question

The second research question asks whether it is better to forecast the concentration and then apply a decision threshold, or to train a classifier on the exceedance label directly. Comparing the two arms by their single best model would be misleading, since the forecast-then-threshold arm has seven regressors to choose from at every budget and the classification arm has three; the better-resourced arm would look stronger for that reason alone, regardless of which strategy is actually better. The comparison below instead matches each classifier to the regressor built from the same underlying algorithm, ridge against the penalised logistic it most closely resembles, and the change-target random forest and gradient boosting models against their own classifier counterparts, so that training objective is the only thing that varies.

Table 4.6 already shows more structure than a coin flip. Across all three matched pairs,

Table 4.6: Forecast-then-threshold against direct classification, matched by base algorithm, at one day ahead. Higher is better; bold marks the better arm in each column.

Alarms per year, $h = 1$	40	60	80	100
Ridge vs. penalised logistic				
forecast then threshold	**0.377**	**0.554**	**0.723**	0.850
direct classification	0.356	0.550	0.721	**0.856**
Random forest (change) vs. its classifier				
forecast then threshold	**0.367**	0.550	**0.729**	**0.867**
direct classification	0.331	0.550	0.719	0.856
Gradient boosting (change) vs. its classifier				
forecast then threshold	**0.379**	**0.554**	0.727	0.854
direct classification	0.323	0.546	0.727	**0.865**

every horizon, and all four alarm budgets, 36 comparisons in total, forecast-then-threshold wins 21, direct classification wins 11, and four are exact ties. That is not the even split a genuinely arbitrary choice would produce, but the margin each time is almost always small: a median difference of 0.011 in hit rate, and a largest difference, across every comparison run, of 0.056. Forecasting the concentration and then thresholding it has a real, modest edge that shrinks steadily as the alarm budget loosens: it wins 8 of 9 comparisons at a budget of 40 and 7 of 9 at 60, but direct classification overtakes it at the loosest budget this report considers, winning 6 of 9 at 100. The edge is also uneven by horizon, narrowing steadily rather than changing hands: forecast-then-threshold wins 8 of 12 matched comparisons at one day ahead, 7 of 12 at two days, and 6 of 12 at three. There is no horizon or budget regime in which direct classification leads at one day or two; it only overtakes once the alarm budget is loose enough that a classifier's extra structure stops mattering.

A plausible reading is that a classifier optimises the exceedance label directly and so should have the advantage, while a regressor optimises a different quantity, squared or absolute error on the concentration, and only inherits its warning-system performance as a side effect of being thresholded well. Table 4.6 suggests that advantage is not realised in practice on a series this size: a classifier trained on roughly the same few thousand rows has little more signal to find in the label than a regressor already has in the continuous value, and the regressor's continuous output carries more information into the threshold sweep than a single probability does, which may be what the small but consistent edge reflects.

Given a modest result rather than a decisive one, the forecast-then-threshold arm is preferred on grounds beyond the numbers in Table 4.6 as well: it produces a concentration, which can be rethresholded for any future health policy without refitting, whereas a classifier is bound to the threshold it was trained on. Both considerations point the same way, which is why this report's own warning-system results elsewhere, including Table 4.7 and the weather comparison in Section 4.1.1, are built on the forecast-then-threshold arm throughout rather than on the classifiers.

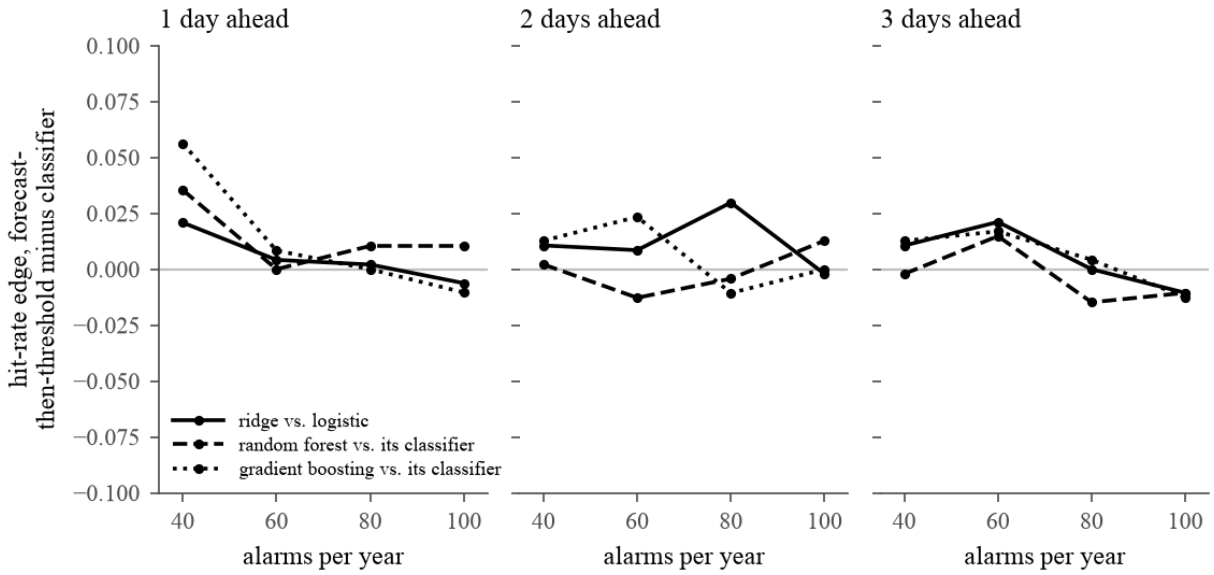


Figure 4.2: Table 4.6 extended to all three horizons: the hit-rate advantage of forecast-then-threshold over its matched classifier, by alarm budget, one line per matched pair. Above the zero line forecast-then-threshold wins; below it the classifier does. The lines start mostly above zero at a budget of 40 and drift toward it, and below it, as the budget loosens and the horizon lengthens, the same narrowing the text above describes as counts rather than as a curve.

4.3 The Decision Threshold

Table 4.7: Best hit rate available within a budget of alarms per year, with the event fixed at the health threshold and only the decision threshold varying.

Alarms per year	40	60	80	100
Best hit rate, $h = 1$	0.379	0.567	0.735	0.881
Best hit rate, $h = 2$	0.366	0.538	0.700	0.840
Best hit rate, $h = 3$	0.359	0.533	0.685	0.827
GRU, 30 day sequence, range over 10 seeds, $h = 1$	0.356–0.377	0.546–0.565	0.719–0.735	0.863–0.881
ARIMA(1,1,1) with Fourier terms, $h = 1$	0.358	0.567	0.727	0.867
Gradient boosting, change target, $h = 1$	0.379	0.554	0.727	0.854
Persistence, $h = 1$	0.371	0.558	0.723	0.873
Climatology, $h = 1$	0.352	0.521	0.660	0.821

Table 4.7 contains the principal result of this project. Moving the alarm budget from 40 to 100 per year changes the achievable hit rate at one day ahead from 0.379 to 0.881. Holding the budget fixed and changing the model instead moves it by around five points.

Two individual comparisons make the point sharper than the aggregate does. At three days ahead and a budget of 40 alarms a year, persistence, a forecast that does nothing but repeat yesterday's reading, is not merely competitive but is the single best model in the entire study, 0.359, ahead of every machine learning regressor, ARIMA, and every one of the recurrent network's ten seeds. At the loosest budget this report considers, 100 alarms a year, and the same horizon, a climatological forecast, which uses no recent information whatever, comes within roughly a hundredth of the best fitted model, 0.816 against 0.827. Figure 4.3 shows the

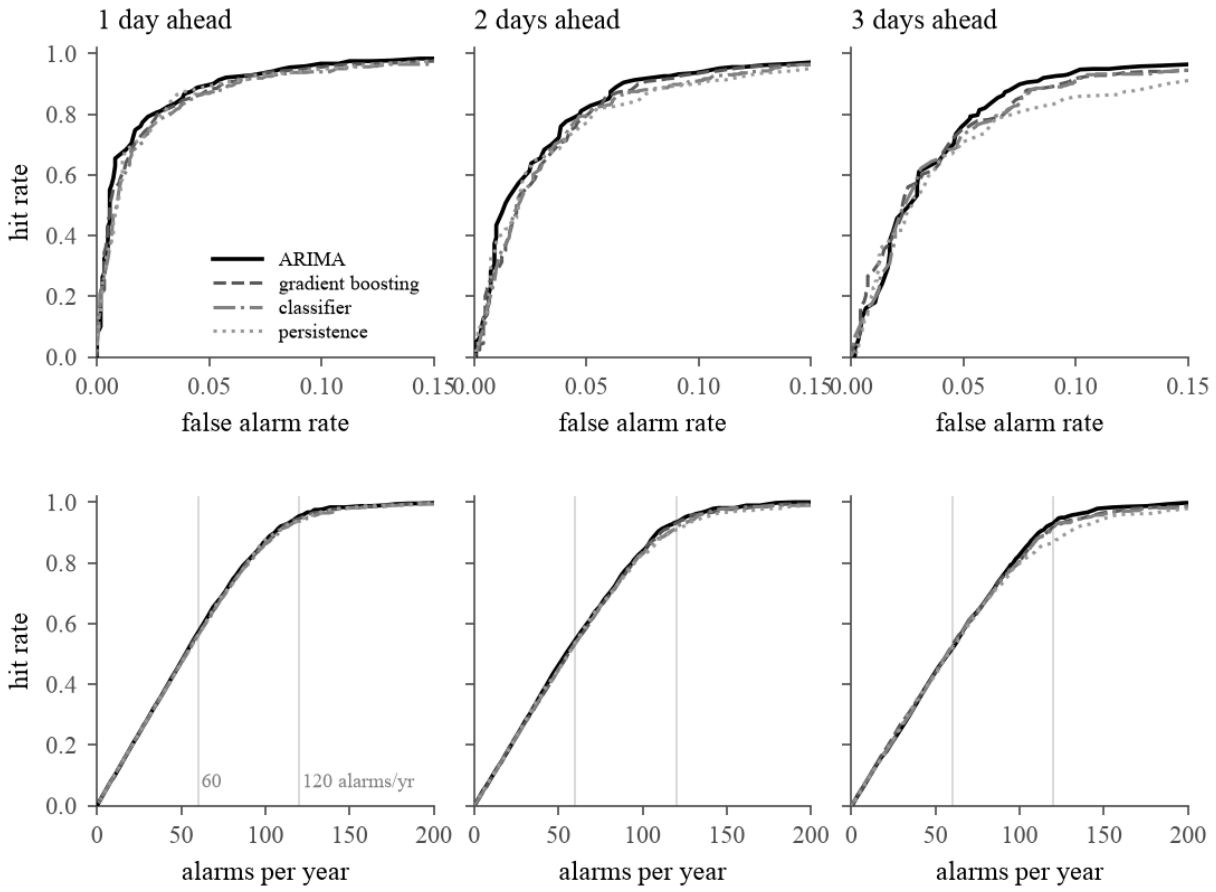


Figure 4.3: The trade-off, by horizon. The top row shows hit rate against false alarm rate; the bottom row replaces the false alarm rate with alarms raised per year, the unit in which an operator budgets. The curves very nearly coincide, which is the finding. They have deliberately not been separated for legibility.

same result as curves; they very nearly coincide, and they have deliberately not been separated for legibility.

The mechanism behind the first of those two comparisons is already visible in Figure 3.2: exceedances arrive as multi-day episodes, not isolated days, so the most informative fact about whether day $t+3$ is hazardous is often simply that day t already is. Persistence is exactly the forecast that encodes this and nothing else, carrying today's reading forward unchanged to every horizon. At a budget this tight, a model may only raise the alarm on the days its own forecast ranks highest, and the safest such day to flag is not the onset of a new episode, which nothing in this suite predicts reliably three days out, but a day already deep inside one, where persistence's assumption of no change is closest to true. The more elaborate models are not wrong to forecast a trajectory rather than a constant; at three days ahead that trajectory is carried mainly by the same seasonal and autoregressive structure persistence ignores entirely, and fitting it introduces exactly the small deviations that can cost a model its place once only the forty most confident days in the whole study are allowed to count.

The reading is not that modelling is pointless. It is that the quantity a decision maker controls, namely how many alarms per year can be justified, dominates the quantity a modeller controls. A report that presents a model comparison without stating the operating point it was evaluated at has answered a question nobody asked.

4.4 Chapter Summary

Table 4.1 answers the first research question with a horizon-dependent result: the deep learning model wins at one day ahead, the classical model wins or ties beyond it. Table 4.4 (Section 4.1.1) shows that boundary holding at one day ahead once observed weather is added, but across ten seeds rather than the original five it leaves two and three days ahead a statistical tie rather than a clean win for either model. Carried through to the decision threshold, Table 4.5 shows that gain buys only a little over a point of hit rate at a fixed alarm budget at most, confirming rather than complicating the answer to the third question. Table 4.6 (Section 4.2) answers the second: matched by algorithm rather than by whichever model happens to be best, forecasting the concentration and then thresholding it beats direct classification in 21 of 36 comparisons to 11, with four ties, a real but modest edge rather than a decisive one. Section 4.3 answers the third, and it is the result the rest of this report is organised around: the alarm budget, not the model, is what principally determines warning performance. Chapter 5 takes up what that means in practice.

Chapter 5

Discussion

5.1 The Operational Question Dominates the Modelling Question

Table 4.7 and Figure 4.3 together make a single point, and this section states what follows from it rather than repeating the numbers. A decision maker who fixes how many alarms per year an operator or the public will tolerate has more influence over how many hazardous days are caught than a modeller choosing among the ten specifications fitted in this project. Moving the budget from 40 to 100 alarms a year moves the achievable hit rate at one day ahead from 0.379 to 0.881, a range of fifty points. Holding the budget fixed and switching the model instead moves the hit rate by about five points, and at three days ahead, at the loosest budget this report considers, a climatology forecast that uses no recent information whatever comes within roughly a hundredth of the best fitted model, 0.816 against 0.827.

This is not an argument against modelling. ARIMA and the fitted networks both earn real, if modest, gains over persistence in Table 4.1, and a better point forecast still narrows the range of thresholds worth considering. It is an argument about where the leverage in this problem actually sits. A hazardous air quality warning system is not a machine learning benchmark with a threshold attached to it afterwards; the threshold is the policy instrument, and the forecasting model only supplies its input. A comparison of models that does not first fix the alarm budget it was evaluated at has answered a question nobody who has to operate the system actually asked. The practical recommendation that follows is to treat the alarm budget as the primary subject of discussion with the health authority commissioning such a system, and the choice of forecasting model as a secondary refinement of a curve whose shape is already mostly fixed by what every model in this study shares: the persistence of concentration from one day to the next, and the annual cycle.

5.2 How Large Is a Difference Between Models?

Three results in this project bound that question from below. Running the identical pipeline on two different machines moved the tree models by up to 0.004 in relative error, because histogram gradient boosting parallelises with OpenMP and floating point summation order depends on the thread count. Changing only the random initialisation of the GRU moved it by 0.028 to 0.048 across ten seeds; the LSTM's own spread, measured the same way, is tighter at every horizon, 0.018 to 0.023, but still far from zero. A third and less expected result bounds it further: rerunning the identical GRU command with the same seed on the same machine moved its score by nearly as much as switching to a genuinely different seed does, which Section 5.3 discusses and which this report takes as the practical floor on how precisely a trained network's result can be stated at all; the same check was not repeated for the LSTM, so whether it shares the GRU's specific reproducibility defect or only its general sensitivity to initialisation is not yet known.

Table 4.1 contains several gaps at or below those two floors, and naming them is more convincing than asserting that they exist. At one day ahead, gradient boosting on the change target scores 0.983 and the random forest on the level target scores 0.988, a gap of 0.004, right at the floor measured on a different model entirely. The same pair sits even closer together at three days, 0.878 against 0.879, a gap of 0.001, well inside that floor rather than merely at it. Neither pair can be ordered with any confidence given a floor measured at that same size on a different model entirely; the measurement was never repeated for every pair in the table, so the honest reading is that it might apply more broadly than the one architecture it was taken on, not that it is known to be absent elsewhere.

The sharpest instance involves the GRU directly, because its own reported range is wide enough to supply the comparison by itself. At three days ahead, its worst seed scores 0.879, a thousandth away from gradient boosting's change-target result of 0.878, several rows lower in the same table. Nothing about the data or the method changed between those two numbers; a different random seed for the same architecture was enough to move it from a result indistinguishable from the classical benchmark, at that same horizon the GRU's best seed ties ARIMA to within a few parts in ten thousand (Section 4.1), to one indistinguishable from the weakest-looking machine learning regressor in the table. A table that printed only one of those two seeds would have told a reader something true about that run and nothing reliable about the architecture.

Several of the gaps between adjacent models in Table 4.1 are of that order even where no seed or machine-level floor was measured directly for that pair. Any claim that one of those models is better than another, made without stating the run to run variation, is not supported by the evidence.

A formal test of the same question, rather than a tolerance measured by rerunning and com-

pared by eye, is available for the one comparison in this report with enough seeds to support it: the network against ARIMA, in Table 4.2 (Section 4.1). It is, throughout, the more conservative reading. At one day ahead the network's advantage reaches conventional significance in four of the ten seeds, not the eight credited to it by the noise-floor count above; at two days none of the ten reaches it, where the noise-floor count had already read most of the table as a tie; at three days a single seed does, and even that one only at $p = 0.049$. The two readings differ because each bounds something different. The noise floor measured here, 0.004 to 0.048, bounds how much a score moves for reasons that have nothing to do with which model is better: thread scheduling, floating point summation order, a different draw of initial weights. The Diebold-Mariano statistic bounds something else entirely, how much the mean of a loss differential computed over roughly 1,700 individually autocorrelated daily forecast errors can vary by chance even when the two models being compared are, in truth, equally good. Both are genuine sources of uncertainty in a result this report already reports as a range rather than a point; the formal test shows the second is generally the larger of the two in this project, which is a reason to read Table 4.1's gaps even more cautiously than the noise floor alone suggests, not a reason to discard the noise floor in favour of the test. The noise floor is measured directly from reruns, where the Diebold-Mariano variance estimate in Equation 3.44 is necessarily a model of the error process rather than a direct measurement of it; a result that failed either bound on its own would be the one actually worth re-examining.

One more layer of uncertainty sits on top of both readings: the seed counts themselves are small-sample proportions, not settled facts. Four of ten seeds reaching significance at one day ahead is itself a point estimate with a wide interval; a Wilson score interval on that proportion runs from 0.17 to 0.69 at the conventional 95% level, and the all-or-nothing counts at two and three days, 0 of 10 and 1 of 10, carry intervals of their own, up to 0.28 and 0.40 respectively. A different batch of ten seeds could plausibly have produced a noticeably different count at any horizon without the underlying comparison having changed at all. This is not a reason to distrust the direction of any of the three findings, which the formal test, the noise floor and the point estimates all agree on, but it is a reason not to treat the exact fraction reported at each horizon as more precise than the sample it was counted from allows.

5.3 Limitations

The analysis rests on a single monitor, OpenAQ location 8118, which Section 3.2 shows is supplied through AirNow and is almost certainly a United States diplomatic post instrument rather than a station in the Central Pollution Control Board network. It was chosen because it is the only nearby monitor with a complete decade of record, against the eighteen months every CPCB station returns; the choice buys history at the cost of representativeness. The absence of a weekly cycle documented in Section 3.3.2 is consistent with a diplomatic enclave rather than

a roadside site, which supports this reading of the limitation rather than leaving it an unrelated coincidence, but it does not change the fact that every number in this report describes one point in the city rather than Delhi as a whole.

The project forecasts one city. Delhi was chosen because its air quality problem is severe and well documented, not because it is representative of Indian cities generally. Pollution sources, meteorology, and the strength of the seasonal cycle that drives the Fourier terms in Section 3.3.2 all vary by region, and a coastal city or one without a dominant biomass burning season could show a different balance between the classical and machine learning models, or a genuine weekly cycle where this station shows none. Nothing in this report should be read as a claim about any station or city other than the one it was fitted to.

The upper cleaning threshold of $1000\,\mu g\,m^{-3}$ is a judgement rather than a fact, and its effect on the reported conclusions was checked directly rather than assumed. Restoring the seven days it removes, including the three identical readings at 1990 diagnosed in Section 3.3.1 as a clipped instrument ceiling, gives 3,100 usable days and 927 exceedance days rather than 3,093 and 920, a small shift. Refitting the full rolling-origin harness on this uncapped series leaves the ARIMA specification essentially unchanged, 0.935 against 0.940 at one day ahead and 0.829 against 0.832 at three days, because working in logarithms and differencing the series absorbs the contamination. The machine learning models are not so protected. Ridge moves from a near tie with persistence to a clear loss at one day ahead, 0.997 to 1.118, and gradient boosting on the level target moves from a narrow win over persistence to a loss at three days, 0.883 to 1.006. The central finding of Research Question 1, that the classical benchmark beats every machine learning model at every horizon, therefore does not merely survive removing the threshold; it strengthens, because the models it beat most narrowly are precisely the ones most exposed to the uncleaned tail. The threshold matters, but it matters more to the machine learning suite than to the model this project ultimately recommends.

Hyperparameters for the tree-based models were chosen at every refit from a small explicit grid, scored on a held-out slice of the training window taken in time order, rather than from an exhaustive or Bayesian search. This followed a real failure during development, in which fixed hyperparameters let gradient boosting overfit badly enough to score worse than persistence; the grid search fixed that specific problem, but a small grid can still miss a configuration that a wider search would find. The machine learning results in Table 4.1 should therefore be read as a lower bound on what those model classes could achieve with more computation, not as their ceiling.

The recurrent network's own architecture, 24 hidden units and a 30-day input window, was fixed by inspection, with no search equivalent to the tree models' grid. A bounded check run after the fact, fitting once on the original training/validation split used before the rolling-origin harness existed rather than refitting the full ten-seed evaluation, varied hidden size and window length one at a time across six settings and three of the ten seeds used elsewhere. At one day

ahead the default is within 0.2 points of relative error of the best setting tried, a longer window; at two days it is the best of the six outright. At three days, the horizon where the classical benchmark already has the clearest edge, the default is the worst of the six: a 45-day window scored 2.2 points better on this check. Whether that gap would survive the full rolling-origin evaluation across all ten seeds is untested, and nothing in Table 4.1 has been changed on the strength of a single-split, three-seed result. The honest reading is that the three-day comparison in this report rests on a configuration this check found a clearly better alternative to, while the one- and two-day comparisons rest on a configuration at or near the best of those tried.

The same bounded method applied to a different setting: holding hidden size and window length at the report's defaults and varying only the number of Fourier harmonics the network receives, from the two pairs used throughout this report to the four ARIMA uses, on the same single-split check and the same three seeds. At one day ahead the difference is negligible, an improvement of 0.07 points. At two days, four harmonics improve the mean by 0.55 points. At three days, the horizon where ARIMA's advantage is otherwise clearest, four harmonics improve the mean by 3.96 points: low enough on this check that the network's mean would sit numerically below ARIMA's reported rolling-origin figure, not above it. This does not show that the ten-seed rolling-origin result at three days would change if the network were refit with four harmonics there too; that run was not done, and a gap this size on one validation split is not automatically the same gap under the full evaluation. It does show that the account above of why ARIMA wins at longer horizons, that the network has to learn seasonality a classical model is simply handed, is not quite right: the network is handed a coarser version of the same signal, and how much of the gap that coarseness explains, as opposed to a genuine representational limit, is now an open and fairly pointed question rather than a closed one.

A third bounded check, raised directly by Dr Tian, varied the optimiser and learning rate the GRU trains with, holding architecture at the report's defaults, on the same single-split, three-seed design as the two checks above. At one and two days ahead the default, Adam at a learning rate of 0.01, is at or near the best setting tried: switching to RMSprop or SGD, or moving the learning rate by a factor of three in either direction, never improves the mean by more than a point. At three days the pattern breaks the same way the window-length check broke it: SGD with momentum improves the mean by roughly two points over the default, 0.839 against 0.858, with a visibly tighter spread across seeds too. The same horizon that already has the worst-tested window length and the most to gain from more Fourier harmonics also has the worst-tested optimiser, three separate bounded checks now agreeing that the three-day GRU configuration used throughout this report is the one this project's own sensitivity analysis would least defend if a reader asked it to.

A full rerun of this project's pipeline, performed late in the writing process specifically to refresh every result from scratch, surfaced a further limitation worth stating plainly rather than leaving implicit in a table footnote. Every classical and tree-based model reproduced its

reported score either exactly (ARIMA, ridge, the benchmarks) or within a few thousandths (random forest, gradient boosting) when refitted on the same machine with the same fixed random seeds. The recurrent network did not: rerunning the identical command with the same `-seed` moved its score by nearly as much as switching to a genuinely different seed does, evidently because training accumulates small floating-point differences from multi-threaded linear algebra over many epochs in a way a single tree fit does not. The practical consequence is that five seeds, this project's original sample, was not enough to characterise the network reliably: the weather-covariate finding at two and three days ahead (Section 4.1.1) looked like a clear win under the first five seeds fitted and looked like a tie or a loss once five more were added. The sample was doubled to ten seeds throughout this report for that reason, and even ten should be read as a reduction in the problem rather than a solution to it; a result reported as a range across seeds is a statement about the spread of outcomes this training procedure produces, not a bound on where its true performance lies.

The LSTM of Section 3.5.2 and the transformer of Section 3.5.3 were each held to the same ten-seed rolling-origin evaluation as the GRU for accuracy, but neither was carried further: neither was given a weather-augmented channel set (Section 3.3.4), and neither was run through the decision-threshold sweep of Section 4.3, so both appear in Table 4.1 and Table 4.2 but nowhere in Chapter 4's warning-system tables. Given the central finding of this report, that the alarm budget dominates which model is chosen far more than the model itself does, there is no particular reason to expect either would disturb that finding; neither was tested directly because the marginal evidentiary value did not justify the computation with submission close, not because a result was anticipated and avoided.

The transformer began, like the GRU's own hyperparameter and optimiser checks above, as a bounded single-split check: a small encoder, sixteen- dimensional embeddings, two attention heads, one encoder layer, sinusoidal positional encoding, fitted on the identical channel window. Section 2.3.1 argues this project's roughly three thousand observations favour a recurrent architecture over one built for self-attention, and on that first single split the transformer did not beat ARIMA's own single-split figures (0.903, 0.850 and 0.798 at one, two and three days, Section 3.6) at any horizon, consistent with the argument, but was more competitive with the GRU on that same comparison than the argument predicted: 0.903, 0.890 and 0.818 against the GRU's single-split 0.914, 0.886 and 0.858, ahead of the GRU at one and three days specifically. Dr Tian's reply to a summary of that result, after the GRU-and-LSTM response was sent, questioned the small-data premise directly rather than accepting the bounded check as the last word: this project's series is roughly four thousand days long counting every evaluation block, which by his own reading of the literature is well above the few hundred observations needed to justify a transformer. The full ten-seed rolling-origin treatment reported in Section 4.1 answers that question directly rather than leaving it to one validation split: the transformer beats the GRU in eight of ten matched seeds at every horizon and is within the LSTM's own seed-to-seed spread,

the strongest of the three networks against ARIMA by the formal significance test at one day ahead. The small-data argument this project made for preferring a recurrent architecture over a transformer is, on this evidence, not well supported either, the same conclusion already reached above for the narrower GRU-over-LSTM parameter-count argument.

A Kolmogorov-Arnold Network was tried as a hybrid specifically because Dr Tian named that combination directly, and remains a bounded check rather than a full evaluation: building it to the transformer's or the LSTM's own standard would be a comparable amount of work, and a bounded check is the way this project has consistently chosen to spend limited time on a question worth a first answer rather than committing to a full one blind. An LSTM backbone handles the sequence, its usual linear output layer replaced by a Kolmogorov-Arnold layer [Liu et al., 2025], a learnable function on each input-output edge rather than a fixed weight. No established, maintained library exists for this the way one does for the GRU, the LSTM or a transformer encoder, so the layer was implemented here from the published construction (a base path plus a B-spline correction on a fixed grid) and checked in isolation before being fitted on real data: a single such layer recovers a held-out $\sin(3x)$ target to within 10^{-4} mean squared error from a near-flat start. The practical risk, training a novel architecture with no library to lean on and no prior use in this project, was real and did not materialise: all three seeds trained stably, with no divergence or non-finite output at any point. On the same bounded single-split check used for the transformer's own first look, the hybrid is competitive with, not dramatically better or worse than, the other three architectures: 0.924, 0.862 and 0.818 at one, two and three days, worst of the four at one day, best of the four at two days, and at three days essentially tied with the transformer's own bounded-check figure, both well ahead of the GRU and LSTM on that specific comparison. A null result here, the hybrid simply failing to train, would have been informative on its own; a working but unremarkable one is the result actually obtained, reported as exactly that rather than dismissed or oversold in either direction. None of this hybrid's bounded numbers has changed anything in Table 4.1; it would need the full rolling-origin treatment, now given to the transformer, before they could.

Wind speed and temperature were added, in Section 3.3.4, but the treatment is narrower than a full meteorological extension in two specific ways that should be stated plainly rather than discovered by a careful reader. First, only observed weather enters the feature table, never a forecast one; a real early-warning system would feed in a numerical weather prediction for the days ahead, which is a materially stronger and untested claim about what is knowable at the forecast origin. Second, boundary layer height, the variable most directly tied to the physical mechanism of a particulate episode, a shallow boundary layer trapping pollutants near the surface, is not available from `meteostat` at all; it would need a reanalysis product such as ERA5, a separate data acquisition problem rather than the free extension this report's own earlier draft assumed it would be. What was carried through, in Section 4.1.1, is the threshold sweep: the accuracy gain from weather survives contact with the decision threshold only partially, buying

a few points of hit rate at some budgets and none at others, which is itself the clearer instance of this report's central finding rather than a loose end.

5.4 Chapter Summary

Section 4.3 (Chapter 4) established that the alarm budget dominates the model; this chapter has argued why that is not an argument against modelling, and has tried to be as honest about the limits of the modelling itself as about its result. Section 5.2 showed that several of the gaps in Table 4.1 are no larger than noise this project measured directly, so a horizon-by-horizon ranking read off that table without the floor in mind would overstate what is known. The Limitations above add to that the single-station and single-city scope, a cleaning threshold that is a judgement rather than a fact, hyperparameter grids that were never exhaustively searched, a weather extension narrower than a full meteorological system, and a recurrent network whose training is not bit-reproducible even at a fixed seed, discovered only because the pipeline was rerun and the result checked rather than assumed. None of these overturns the central finding; several of them, the noise floor and the seed-count doubling most directly, are the reason that finding can be stated with the confidence it is.

Chapter 6

Conclusion

6.1 Summary of the Study

This project forecast hazardous PM2.5 days at a single Delhi monitor one to three days ahead, comparing a classical benchmark against three machine learning regressors and three deep learning networks, and evaluating every specification as a warning system rather than by average error alone. Two real defects in the source data, a clipped instrument ceiling and a mischaracterised coverage statistic, were found and corrected before any model was fitted, and an assumption carried over from the approved proposal, that the series holds a weekly cycle alongside its annual one, was tested directly and found false.

6.2 Main Conclusions

Across ten models spanning simple benchmarks, a classical time series model, three machine learning regressors, and a recurrent network, the spread in achievable hit rate at a fixed alarm budget is about five percentage points; the spread across alarm budgets, holding the model fixed, is about fifty. The choice a decision maker makes about how many alarms per year can be justified therefore has far more influence on how many hazardous days are caught than the choice a modeller makes among the specifications fitted here.

Machine learning earned its place only partially. ARIMA with annual Fourier terms beats every machine learning regressor at every horizon, now backed by a Diebold-Mariano test at every one of those comparisons rather than point estimates alone (Section 4.1), a result that strengthens rather than weakens under the sensitivity analysis in Section 5.3. Among models using the series' own history alone, the fitted networks are the ones that win, and only at one day ahead, each reported honestly as a range across ten random initialisations rather than as a single favourable seed. Of the three, the LSTM beats the GRU at every horizon despite

carrying roughly a third more parameters, directly against the parameter-count reasoning that motivated choosing the smaller architecture in the first place (Section 2.3), and the transformer, added only after Dr Tian questioned the small-data argument against it directly, beats the GRU in eight of ten matched seeds at every horizon and matches the LSTM, carrying the strongest formal significance of the three against ARIMA at one day ahead (Section 4.1). A Kolmogorov-Arnold-LSTM hybrid, tried afterward as a bounded check rather than a full evaluation because no established library exists for it, was more competitive with the other three architectures on that bounded comparison than a novel, untried component might suggest, without closing the gap to ARIMA. Adding wind speed and temperature (Section 4.1.1) does not move that boundary as cleanly as a first batch of five seeds suggested: across ten seeds the weather-enabled network still wins clearly at one day ahead, but at two days it is tied with ARIMA within this report's own noise floor for six of ten seeds and ahead for only three, and at three days it is more often a clear loss than a win, five of ten against three. Doubling the seed count is what surfaced this, and Section 5.3 discusses why the network's training is the one part of this project's methodology where a fixed seed does not guarantee a fixed answer. Gradient boosting improves at every horizon with the same weather covariates, though the random forest does not, so the gain is specific to which model and which architecture, not a blanket effect of adding more inputs. Carried through the decision threshold, that accuracy gain buys at most a little over a point of hit rate at a fixed alarm budget and nothing at all at most of them, which is this report's central finding recurring at a more demanding test: even a forecaster that looked materially more accurate under an initial, smaller sample does not escape the ceiling the alarm budget imposes.

On the second research question, forecasting the concentration and then thresholding it beats training a classifier directly on the exceedance label, but only modestly: matched by algorithm rather than by whichever model happens to be best, it wins 21 of 36 comparisons to 11, with four ties, a real lean rather than the coin flip a single qualitative judgement would suggest, yet still small enough in each individual case, typically a little over one point of hit rate, that either arm is defensible. The forecast-then-threshold arm is used throughout this report's own warning-system results for a reason beyond that narrow margin: it yields a concentration that can be rethresholded for any future health policy without refitting, while a classifier is welded to the threshold it was trained on.

The supervisor's instruction to try several methods rather than defaulting to ARIMA was the right one: it is what makes the negative result at two and three days credible, and what surfaces the one day ahead result that a project restricted to classical methods would never have found.

6.3 Future Work

Six extensions follow: two from where this report's own weather work stopped, a third from the Fourier-harmonic check in Section 5.3, a fourth from the single-station limitation, a fifth from the scope boundary the LSTM and the transformer now share, and a sixth from the one architecture that remains a bounded check after Dr Tian's own suggestion. The threshold sweep has already been put through the weather-augmented models (Section 4.1.1), so the open thread is not whether that comparison happened but what would change its answer: a real forecast weather product, a numerical weather prediction for the days ahead rather than the conditions already measured, is the stronger and more useful claim for an operational system, and the one this project deliberately did not make. A second is boundary layer height, unavailable from `meteostat`, which would need a reanalysis product such as ERA5 and is the single variable most directly tied to the physical mechanism of a particulate episode rather than a correlate of it; whether it would move the threshold-sweep result more than wind speed and temperature did is an open question precisely because the gain so far has been small.

A third, separate from either of those, follows directly from Section 5.3's Fourier-harmonic check: refitting the recurrent network with four harmonics rather than two under the full ten-seed rolling-origin evaluation, rather than the single validation split used so far, would settle whether the gap to ARIMA at two and three days ahead is substantially a consequence of that one previously undocumented difference in Methodology's own specification, or whether the bounded check's result does not survive a more demanding evaluation.

A fourth extension is independent of weather: repeating the analysis at a second, independent station, ideally one inside the Central Pollution Control Board network once its record lengthens, would test whether the findings here are a property of Delhi air quality or an artefact of the one diplomatic enclave monitor this project was restricted to.

A fifth extension completes the LSTM's and the transformer's own treatment: fitting each with the weather channels of Section 3.3.4 and carrying each through the decision-threshold sweep of Section 4.3, the two steps Section 5.3 states were not taken for either. Given how narrowly the weather extension and the GRU-versus-LSTM comparison have each moved the central finding on their own, the honest expectation is that this would refine rather than overturn it for either network. A stated expectation is not the same evidence as the run itself, though.

A sixth follows directly from the one architecture that remains a bounded check in Section 5.3: a full ten-seed rolling-origin evaluation of the Kolmogorov-Arnold-LSTM hybrid, in place of the single-split, three-seed check it was given here, the same promotion the transformer itself was given in this report once its own bounded check and Dr Tian's challenge to the small-data reasoning behind it made a fuller treatment worth the computation. The hybrid was more competitive on its bounded check than a novel, library-free component might suggest, competitive with all three other architectures at two and three days ahead specifically; whether that holds

under the full evaluation the transformer itself was just given is precisely what this extension would decide.

Until any of these six is carried out, the central recommendation stands on its own: a hazardous air quality warning system should be designed around the alarm budget its operator can justify, with the forecasting model treated as a secondary refinement rather than the primary design choice.

6.4 Chapter Summary

Three research questions were asked and three were answered. Machine learning earns a real but narrow place: only the fitted networks beat the classical benchmark, and only one day ahead, with the LSTM ahead of the GRU despite more parameters, directly against the reasoning that motivated choosing the smaller one, and the transformer, tested only after that same kind of reasoning was challenged directly, competitive with both. Forecasting the concentration and then thresholding it has a modest, genuine edge over classifying the exceedance directly, not a decisive one. And the decision threshold, not the model, is what principally determines how many hazardous days a warning system actually catches, a finding that held when tested against a cleaning-threshold sensitivity check, against a materially more accurate weather-augmented forecaster, and against a doubled sample of the one model whose own training turned out not to be fully reproducible. A project that set out to compare machine learning against classical time series methods for a public health problem ends by recommending that the comparison matters less than where the operator sets the alarm.

Bibliography

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.

Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012. doi: 10.1016/j.ins.2011.12.028.

George E. P. Box, Gwilym M. Jenkins, and Gregory C. Reinsel. *Time Series Analysis: Forecasting and Control*. Wiley, 4th edition, 2008. doi: 10.1002/9781118619193.

Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996. doi: 10.1007/BF00058655.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Routledge, 2017. doi: 10.1201/9781315139470.

Central Pollution Control Board. National air quality index. Control of Urban Pollution Series CUPS/82/2014-15, Ministry of Environment, Forest and Climate Change, Government of India, 2014. URL `https://cpcb.nic.in/displaypdf.php?id=bmF0aW9uYWwtYWlyLXF1YWxpdHktaW5kZXgvRklOQUwtUkVQT1JUX0FRSV8ucGRm`.

Zhengping Che, Sanjay Purushotham, Kyunghyun Cho, David Sontag, and Yan Liu. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085, 2018. doi: 10.1038/s41598-018-24271-9.

Jie Chen and Gerard Hoek. Long-term exposure to PM2.5 and all-cause and cause-specific mortality: A systematic review and meta-analysis. *Environment International*, 143:105974, 2020. doi: 10.1016/j.envint.2020.105974.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. doi: 10.3115/v1/d14-1179.

Daniel H. Cusworth, Loretta J. Mickley, Melissa P. Sulprizio, Tianjia Liu, Miriam E. Marlier, Ruth S. DeFries, Sarath K. Guttikunda, and Pawan Gupta. Quantifying the influence of agricultural fires in northwest india on urban air pollution in Delhi, India. *Environmental Research Letters*, 13(4):044018, 2018. doi: 10.1088/1748-9326/aab303.

Alysha M. De Livera, Rob J. Hyndman, and Ralph D. Snyder. Forecasting time series with complex seasonal patterns using exponential smoothing. *Journal of the American Statistical Association*, 106(496):1513–1527, 2011. doi: 10.1198/jasa.2011.tm09771.

Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995. doi: 10.1080/07350015.1995.10524599.

Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997. doi: 10.1006/jcss.1997.1504.

Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001. doi: 10.1214/aos/1013203451.

Sarath K. Guttikunda, Rahul Goel, and Pallavi Pant. Nature of air pollution, emission sources, and management in the indian cities. *Atmospheric Environment*, 95:501–510, 2014. doi: 10.1016/j.atmosenv.2014.07.006.

David Harvey, Stephen Leybourne, and Paul Newbold. Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2):281–291, 1997. doi: 10.1016/S0169-2070(96)00719-4.

Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

Hansika Hewamalage, Klaus Ackermann, and Christoph Bergmeir. Forecast evaluation for data scientists: Common pitfalls and best practices. *Data Mining and Knowledge Discovery*, 37 (2):788–832, 2023. doi: 10.1007/s10618-022-00894-5.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.

Rob J. Hyndman and Yeasmin Khandakar. Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3), 2008. doi: 10.18637/jss.v027.i03.

Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006. doi: 10.1016/j.ijforecast.2006.03.001.

Hiren Jethva, Omar Torres, Robert D. Field, Alexei Lyapustin, Ritesh Gautam, and Vinay Kayetha. Connecting crop productivity, residue fires, and air quality over northern india. *Scientific Reports*, 9(1):16594, 2019. doi: 10.1038/s41598-019-52799-x.

Ningbo Jiang, Rinat Akter, Glenn Ross, Stephen White, John Kirkwood, Gunaratnam Gunashanhar, Scott Thompson, Matthew Riley, and Merched Azzi. On thresholds for controlling negative particle (PM2.5) readings in air quality reporting. *Environmental Monitoring and Assessment*, 195(10):1187, 2023. doi: 10.1007/s10661-023-11750-4.

Ian T. Jolliffe and David B. Stephenson, editors. *Forecast Verification: A Practitioner's Guide in Atmospheric Science*. Wiley, 2nd edition, 2012.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021. doi: 10.1098/rsta.2020.0209.

Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021. doi: 10.1016/j.ijforecast.2021.03.012.

Roderick J. A. Little and Donald B. Rubin. *Statistical Analysis with Missing Data*. Wiley, 3rd edition, 2019. doi: 10.1002/9781119482260.

Shuai Liu, Hong Ji, and Morgan C. Wang. Nonpooling convolutional neural network forecasting for seasonal time series with trends. *IEEE Transactions on Neural Networks and Learning Systems*, 31(8):2879–2888, 2020. doi: 10.1109/TNNLS.2019.2934110.

Ziming Liu, Max Tegmark, Pingchuan Ma, Wojciech Matusik, and Yixuan Wang. Kolmogorov-Arnold networks meet science. *Physical Review X*, 2025. doi: 10.1103/4t7t-v19l.

Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3):e0194889, 2018. doi: 10.1371/journal.pone.0194889.

Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting*, 36(1): 54–74, 2020. doi: 10.1016/j.ijforecast.2019.04.014.

Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4): 1346–1364, 2022. doi: 10.1016/j.ijforecast.2021.11.013.

Adil Masood and Kafeel Ahmad. A model for particulate matter (PM2.5) prediction for Delhi based on machine learning approaches. *Procedia Computer Science*, 167:2101–2110, 2020. doi: 10.1016/j.procs.2020.03.258.

Donald M. Miller. Reducing transformation bias in curve fitting. *The American Statistician*, 38 (2):124–126, 1984. doi: 10.1080/00031305.1984.10483180.

C. Arden Pope and Douglas W. Dockery. Health effects of fine particulate air pollution: Lines that connect. *Journal of the Air & Waste Management Association*, 56(6):709–742, 2006. doi: 10.1080/10473289.2006.10464485.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.

Joseph T. Schaefer. The critical success index as an indicator of warning skill. *Weather and Forecasting*, 5(4):570–575, 1990. doi: 10.1175/1520-0434(1990)005<0570:TCSIAA>2.0.CO;2.

Sima Siami-Namini, Neda Tavakoli, and Akbar Siami Namin. A comparison of ARIMA and LSTM in forecasting time series. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 1394–1401, 2018. doi: 10.1109/ICMLA.2018.00227.

Sandhya Singh and Arun Srivastava. Machine learning approach to PM2.5 forecasting and health risk assessment during stubble burning period in Delhi. *Aerosol Science and Technology*, 59(11), 2025. doi: 10.1080/02786826.2025.2513503.

Leonard J. Tashman. Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4):437–450, 2000. doi: 10.1016/S0169-2070(00)00065-0.

G. Peter Zhang. Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50:159–175, 2003. doi: 10.1016/S0925-2312(01)00702-0.

Guoqiang Zhang, B. Eddy Patuwo, and Michael Y. Hu. Forecasting with artificial neural networks: The state of the art. *International Journal of Forecasting*, 14(1):35–62, 1998. doi: 10.1016/S0169-2070(97)00044-7.